



概念&逻辑结构设计

课题名称： 酒店管理系统

课题组员： 周旻旻 程什开 陈婧琳 陈雨杰

完成时间： 5.15

分工：

本章按照系统功能模块分工：

客房资源管理-周旻旻 支付财务管理-程什开

后台管理-陈婧琳 预定入住管理-陈雨杰

排版整合：周旻旻

4 概念结构设计

4.1 客房资源管理

1. 酒店实体

属性：酒店编号、创建时间、酒店名称、酒店类别、酒店星级、酒店图片、酒店地址、服务热线、酒店简介

作用：记录酒店的基础展示信息和维护信息，是客房目录的归属对象，用于前台酒店展示和后台酒店资料管理。

2. 房型实体

属性：房型编号、创建时间、房型名称

作用：统一维护客房类型，如大床房、双床房、套房等，支持客房分类、筛选和统计。

3. 客房目录实体

属性：客房目录编号、创建时间、酒店编号、房型编号、客房名称、客房介绍、点击量、评分统计值、基础价格、客房图片、客房设施

作用：记录面向销售和展示的一类客房信息，不表示具体房间，而表示“可销售的客房类型/目录”，用于客房浏览、价格展示和库存统计。

4. 实体房间实体

属性：房间编号、所属客房目录编号、房间号、楼层号、当前状态、创建时间、更新时间

作用：记录酒店中可以实际分配入住的具体房间，是预订、入住、退房、清洁和维修等房态管理操作的直接对象。

5. 价格策略实体

属性：策略编号、所属客房目录编号、策略名称、开始日期、结束日期、价格倍率、优先级、是否启用

作用：记录客房在不同时间段或经营场景下的价格调整规则，用于在基础价格之上计算动态房价。

6. 房态日志实体

属性：日志编号、房间编号、原状态、新状态、来源类型、来源业务编号、操作人编号、备注、变更时间

作用：记录具体房间状态变化过程，如空闲、已预订、已入住、清洁中、维修中等状态转换，支持房态追踪和业务审计。

7. 订单房间明细实体（酒店预订模块）

属性：明细编号、订单编号、房间编号、锁定房价、入住开始日期、入住结束日期、房间订单状态、创建时间

作用：作为订单与实体房间之间的中间实体，解决“一张订单可以预订多个房间、一个房间在不同时间可以出现在多个订单中”的多对多关系。

8. 清洁任务

属性：id、房间编号、退房记录编号、分配员工编号、任务类型、任务状态、计划时间、完成时间、备注、创建时间、更新时间

作用：记录房间清洁任务。当退房、日常清洁或维修完成后需要清洁时，生成清洁任务，并推动房间状态在 **cleaning** 与 **available** 之间变化。

9. 维修记录

属性：id、房间编号、报修员工编号、处理员工编号、问题描述、维修状态、开始时间、结束时间、维修费用、备注

作用：记录房间维修业务。当房间报修后，房态变为 **maintenance**；维修完成后，房间通常转入 **cleaning**，等待清洁完成后恢复可用。

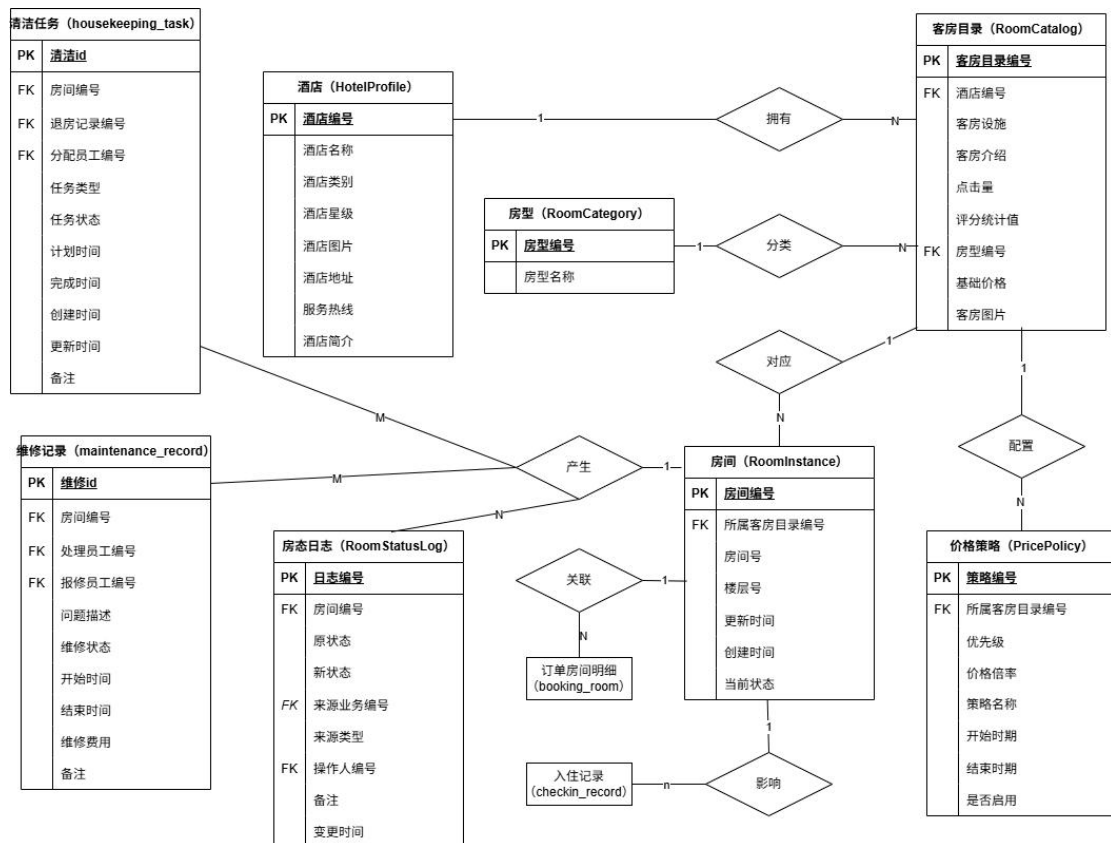


图 1 客房资源管理 ER 图

注：其中来源业务编号 不一定只指向订单，也可能来自 booking_room、入住记录、清洁任务、维修记录等，“FK”作为逻辑外键。

4.2 预定入住

预订与入住模块主要负责客户预订、订单状态维护、员工办理入住、入住人登记以及房态同步等业务。重点围绕以下业务展开：

1. 客户如何创建预订订单；
2. 预订订单如何关联房间；
3. 员工如何根据订单办理入住；
4. 入住记录如何保存入住人信息；
5. 房间状态如何随业务变化而更新。

因此，本模块主要涉及客户、预订订单、入住记录、入住人、员工和具体房间等实体。

1. 客户实体 (customer)

主要属性：customer_id (客户编号) customer_name (客户姓名) phone (联系电话) id_card (身份证号) register_time (注册时间)

业务说明：一个客户可以创建多个预订订单。

2. 预订订单实体 (booking_order)

主要属性：booking_id (预订编号) customer_id (客户编号) room_id (房间编号) checkin_date (入住日期) checkout_date (离店日期) booking_amount (预订金额) booking_status (预订状态) create_time (创建时间)

业务说明：一个预订订单对应一个客户和一个具体房间。

3. 预订状态日志实体 (booking_status_log)

主要属性：log_id (日志编号) booking_id (预订编号) old_status (原状态) new_status (新状态) change_time (变更时间)

业务说明：一个预订订单可以产生多条状态日志。

4. 具体房间实体 (room_instance)

主要属性：room_id (房间编号) room_no (房号) floor_no (楼层) room_status (房态)

业务说明：一个房间可以被多个历史订单使用。

5. 入住记录实体 (checkin_record)

主要属性：checkin_id (入住编号) booking_id (预订编号) room_id (房间编号) employee_id (办理员工编号) checkin_time (入住时间) checkout_time (退房时间) checkin_status (入住状态)

业务说明：一个预订订单最多生成一个入住记录。

6. 入住人明细实体 (checkin_guest)

用于保存实际入住人员信息。

主要属性：guest_id (明细编号) checkin_id (入住编号) guest_name (入住人姓名) guest_id_card (身份证号) guest_phone (联系电话)

业务说明：一个入住记录可以包含多个入住人。

7. 员工实体 (employee)

主要属性：employee_id (员工编号) employee_name (员工姓名) phone (联系电话) position (职位)

业务说明：一个员工可以办理多个入住业务。

本模块中的实体联系如下：

实体联系	联系类型	说明
customer — booking_order	一对多	一个客户可创建多个订单
booking_order — booking_status_log	一对多	一个订单可对应多条状态日志
room_instance — booking_order	一对多	一个房间可对应多个历史订单
booking_order — checkin_record	一对一	一个订单最多对应一个入住记录
checkin_record — checkin_guest	一对多	一个入住记录可对应多个入住人
employee — checkin_record	一对多	一个员工可办理多个入住业务
room_instance — checkin_record	一对多	一个房间可产生多个入住记录

表 1 预定入住实体联系

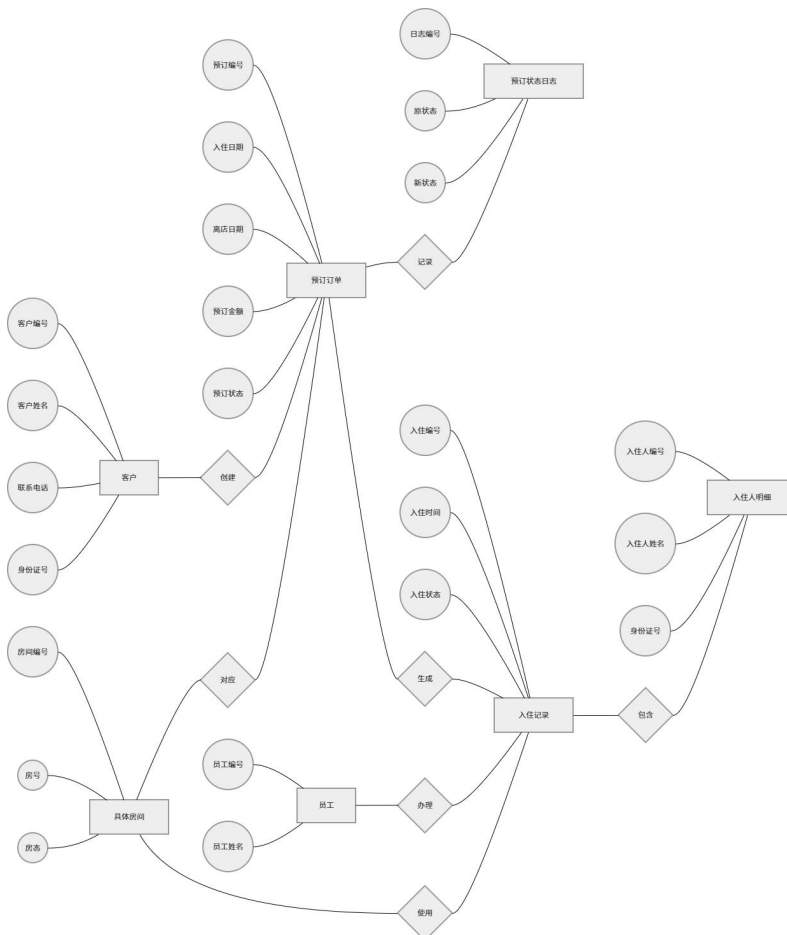


图 2 预定入住 ER 图

4.3 财务结算与支付退款

本局部包含的核心实体有：bill（账单）、payment_record（支付记录）、refund_record（退款记录）。它们与系统的其他核心实体 checkin_record（入住记录）和 booking_order（预订订单）存在紧密的关联。

1. 账单实体

属性：账单编号、房间费用、服务费用、折扣金额、应付金额、已付金额、累计退款、账单状态、创建时间、更新时间

作用：记录客房交易的最终结算汇总信息，通过触发器动态累加支付与退款金额，实现自动化的账单状态变更与财务核销。

2. 支付记录实体

属性：支付编号、支付流水号、关联账单编号、支付方式、支付金额、支付状态、备注、支付时间

作用：记录客户针对账单执行的每一笔实际缴费操作，作为支付触发器的输入源，用于驱动账单“已付金额”的更新及状态转换。

3. 退款记录实体

属性：退款编号、退款流水号、关联账单编号、退款金额、退款原因、退款状态、备注、创建时间

作用：记录因订单取消或账单多付产生的资金原路返回信息，用于抵扣账单累计金额并支持财务审计与原由追溯。

关联实体	关联实体	关系类型	关联外键	业务规则描述
预订订单	账单	1:N	预订编号	一个预订订单在入住消费后可以生成一张或多张账单，但每张账单只属于一个预订订单。
入住记录	账单	1:N	入住编号	一次入住记录在退房结算时生成结算账单。账单中的预订编号和入住编号至少有一个非空。
账单	支付记录	1:N	账单编号	一张账单可通过多次支付来结清（如押金、尾款、分次刷卡）。一笔支付流水只能属于一张账单。
支付记录	退款记录	1:N	支付编号	一笔成功的支付记录在发生退款时，可以产生多次部分退款，且必须关联原支付记录以实现“原路返回”。
客户	账单	1:N	客户编号	一个客户可以拥有多张账单，但每张账单只能唯一地属于一个客户。

表 2 支付模块关联

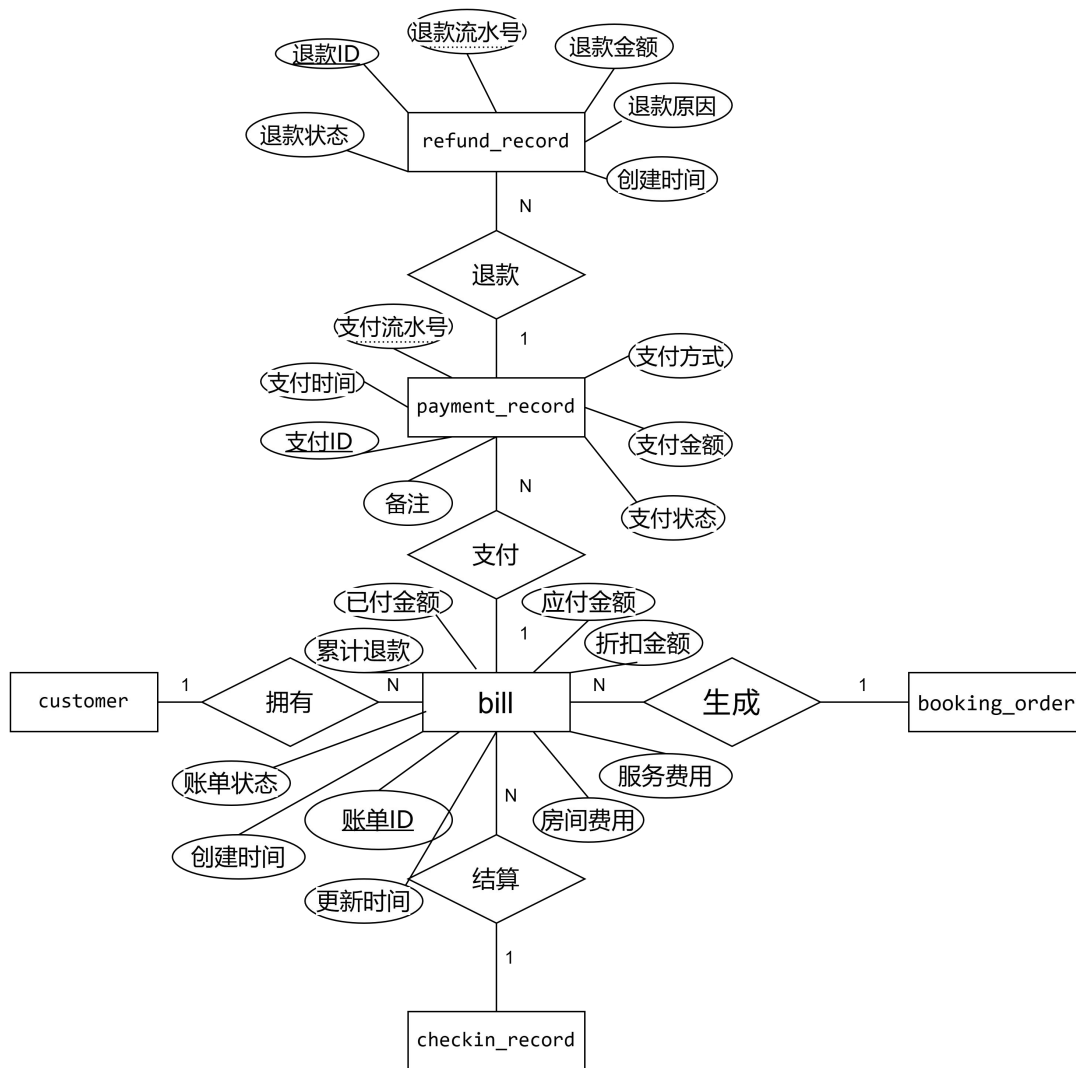


图 3 支付退款管理 ER 图

4.4 后台管理

1. 管理员实体

实体名：管理员

属性：id、账号、密码、角色、创建时间

作用：系统最高权限管理者，负责系统配置、公告管理、数据审核。

2. 员工实体

实体名：员工

属性：id、工号、姓名、密码、电话、职位、入职时间

作用：负责酒店入住办理、订单分配、客房管理、保洁维修任务派发。

3. 用户实体

属性：id、账号、姓名、密码、性别、年龄、电话

作用：系统使用者，可完成注册登录、客房预订、订单支付、评价、收藏等操作。

4. 评价实体（分为酒店评价、客房评价两类）

属性：id、创建时间、关联 ID、用户 ID、评论内容、管理员回复、评分

作用：记录用户对酒店与客房的评价，支持评分统计。

5. 收藏实体

属性：id、用户 ID、关联 ID、关联表名、收藏名称、图片

作用：用户可收藏酒店或客房，支持快速访问。

6. 操作日志实体

属性：id、操作者类型、操作者 ID、业务类型、操作行为、操作时间、备注

作用：记录管理员、员工、用户的所有关键操作，用于审计与追溯。

7. 员工表现实体

属性：id、员工 ID、考核人 ID、考核开始时间、考核结束时间、入住办理数、账单开具数、支付处理数、保洁任务数、维修任务数、操作日志数、管理员评分、综合总分、绩效等级、备注、创建时间

作用：用于员工绩效考核统计，生成员工绩效报表，支撑绩效视图与存储过程。

8. 登录令牌实体

属性：id、用户 ID、用户名、关联表名、角色、令牌、创建时间、过期时间

作用：用于管理员、员工、用户登录鉴权，保证系统登录安全性。

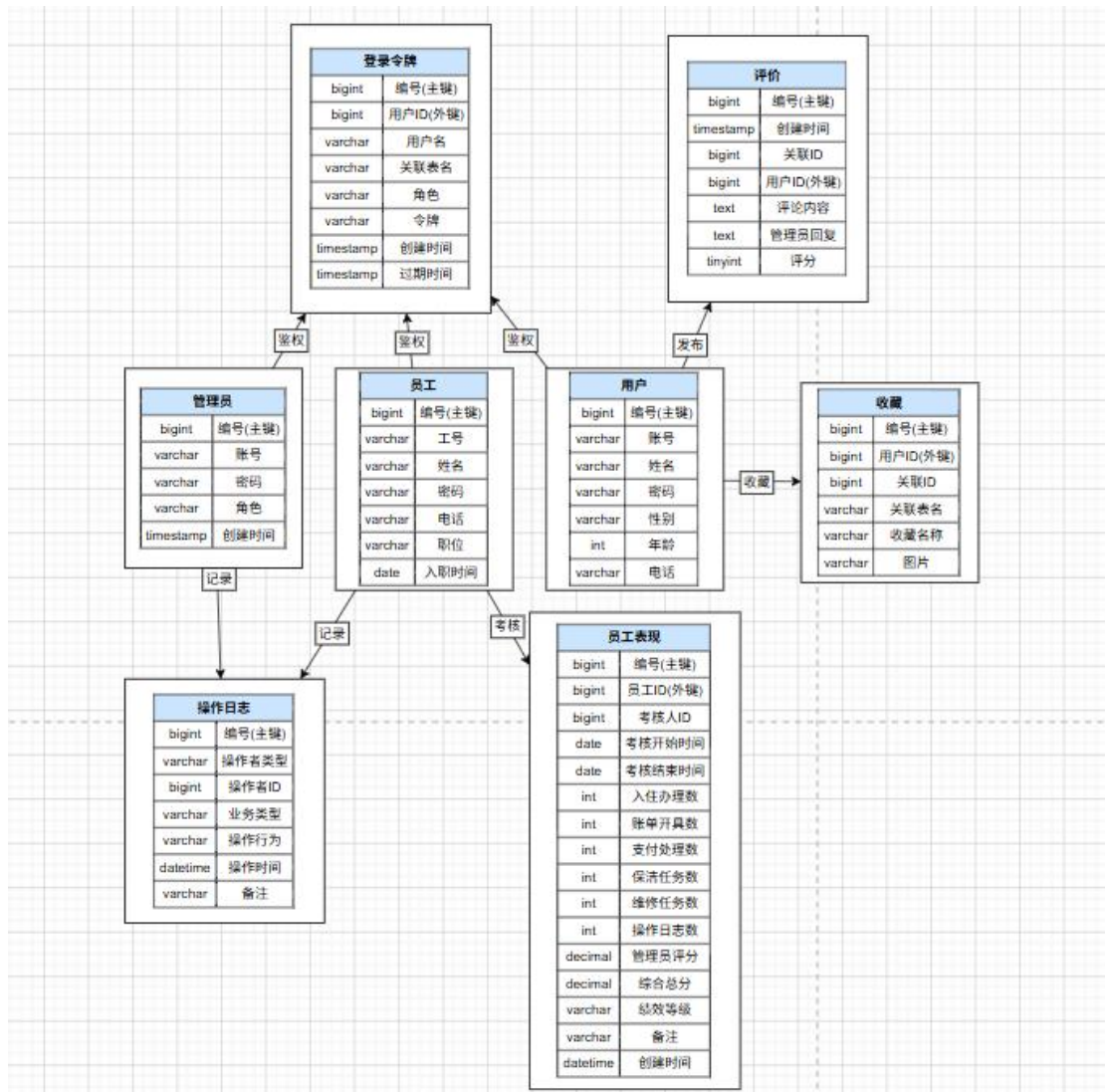


图 4 后台管理 ER 图

5 逻辑结构设计

5.1 逻辑结构设计

5.1.1 客房资源管理模块

1. hotel_profile (酒店信息表)

hotel_profile (id, jiudianmingcheng, leibie, xingji, jiudiandizhi)

主键 PK: id

2. room_category (客房类型字典表)

room_category (id, room_category)

主键 PK: id

候选键 CK: room_category (唯一约束)

3. room_catalog (客房目录表)

room_catalog (id, hotel_profile_id, room_category_id, kefangmingcheng, kefangjiage, shuliang, kefangsheshi, kefangjieshao)

主键 PK: id

外键 FK:

hotel_profile_id 引用 hotel_profile (id), 约束名 fk_catalog_hotel, 级联更新(ON UPDATE CASCADE)

room_category_id 引用 room_category (id), 约束名 fk_catalog_category, 级联更新(ON UPDATE CASCADE)

4. room_instance (实体房间表)

room_instance (id, room_catalog_id, room_no, floor_no, status)

主键 PK: id

候选键 CK: room_no (唯一约束)

外键 FK: room_catalog_id 引用 room_catalog (id), 约束名 fk_room_instance_catalog, 级联更新(ON UPDATE CASCADE)

5. price_policy (价格策略表)

price_policy (id, room_catalog_id, policy_name, start_date, end_date, price_multiplier, priority)

主键 PK: id

外键 FK: room_catalog_id 引用 room_catalog (id),

约束名 fk_price_policy_catalog, 级联更新与删除(ON UPDATE CASCADE, ON DELETE CASCADE)

6. room_status_log (房态变更日志表)

room_status_log (id, room_id, old_status, new_status, source_type, source_id, remark)

主键 PK: id

外键 FK: room_id 引用 room_instance (id), 约束名 fk_status_log_room, 删除时置空(ON DELETE SET NULL)

7. booking_order (预订主单表)

booking_order (id, customer_id, room_catalog_id, room_instance_id, expected_checkin_date, expected_checkout_date, booking_status)

主键 PK: id

外键 FK:

customer_id 逻辑引用 customer (id), 由应用层保证完整性

room_catalog_id 引用 room_catalog (id), 约束名 fk_booking_catalog

room_instance_id 引用 room_instance (id), 约束名 fk_booking_room_instance

8. booking_room (订单房间明细表)

booking_room (id, booking_id, room_id, room_price, room_status)

主键 PK: id

候选键 CK: (booking_id, room_id) 联合唯一

外键 FK:

booking_id 引用 booking_order (id), 约束名 fk_booking_room_order, 级联删除

room_id 引用 room_instance (id), 约束名 fk_booking_room_detail

9. housekeeping_task (清扫任务表)

housekeeping_task (id, room_id, checkin_id, assigned_employee_id, task_type, task_status, scheduled_time, completed_time)

主键 PK: id

外键 FK:

room_id 引用 room_instance (id), 约束名 fk_task_room

checkin_id 逻辑引用 checkin_record (id)

assigned_employee_id 逻辑引用 employee (id)

10. maintenance_record (维修记录表)

maintenance_record (id, room_id, reported_by_employee_id, handled_by_employee_id, issue_desc, maintenance_status, start_time, end_time, cost_amount)

主键 PK: id

外键 FK: room_id 引用 room_instance (id), 约束名 fk_maintenance_room

可以大致分为三层:

1) 酒店与房型基础层: `hotel\_profile`、`room\_category`

2) 客房定义与资源层: `room\_catalog`、`room\_instance`、`price\_policy`

3) 房态联动与过程层: `room\_status\_log`、`booking\_order`、`booking\_room`、`housekeeping\_task`、`maintenance\_record`

5.1.2 预定入住模块

1. customer (客户表)

customer(customer_id, customer_name, phone, id_card, register_time)

主键 PK: customer_id

2. room_instance (具体房间表)

room_instance(room_id, room_no, floor_no, room_status)

主键 PK: room_id

3. booking_order (预订订单表)

booking_order(booking_id, customer_id, room_id, checkin_date, checkout_date, booking_amount, booking_status, create_time)

主键 PK: booking_id

4. booking_status_log (预订状态日志表)

booking_status_log(log_id, booking_id, old_status, new_status, change_time)

主键 PK: log_id

5. employee (员工表)

employee(employee_id, employee_name, phone, position)

主键 PK: employee_id

6. checkin_record (入住记录表)

checkin_record(checkin_id, booking_id, room_id, employee_id, checkin_time, checkout_time, checkin_status)

主键 PK: checkin_id

7. checkin_guest (入住人明细表)

checkin_guest(guest_id, checkin_id, guest_name, guest_id_card, guest_phone)

主键 PK: guest_id

5.1.3 财务结算支付退款模块

1. bill (账单信息表)

bill(id, booking_id, checkin_id, customer_id, room_amount, service_amount, discount_amount, payable_amount, paid_amount, refund_amount, status, created_at, updated_at)

主键 PK: id

外键 FK:

booking_id 引用 booking_order(id), 约束名 fk_bill_booking, 删除时置空(ON DELETE SET NULL)

checkin_id 引用 checkin_record(id), 约束名 fk_bill_checkin, 删除时置空(ON DELETE SET NULL)

customer_id 当前版本未添加物理外键约束, 但逻辑上引用 customer(id), 由应用层保证参照完整性

业务规则: payable_amount = room_amount + service_amount - discount_amount。

paid_amount 不能超过 payable_amount。

2. payment_record (支付记录表)

payment_record (id, bill_id, payment_no, pay_method, pay_amount, pay_status, paid_at, remark)

主键 PK: id

候选键 CK: payment_no

外键 FK: 外键 FK: bill_id 引用 bill (id), 约束名 fk_payment_record_bill, 级联更新与删除(ON UPDATE CASCADE, ON DELETE CASCADE)

3. refund_record (退款记录表)

refund_record (id, payment_id, refund_no, refund_amount, refund_reason, refund_status, created_at)

主键 PK: id

候选键 CK: refund_no

外键 FK: payment_id 引用 payment_record (id),
约束名 fk_refund_record_payment, 级联更新与删除 (ON UPDATE CASCADE, ON DELETE CASCADE)

5.1.4 后台管理模块

1. admin_user (管理员表)

admin_user (id, username, password, role, addtime)

主键 PK: id

2. employee (员工表)

employee (id, employeegonghao, employeexingming, mima, lianxidianhua, zhiwei, addtime)

主键 PK: id

候选键 CK: employeegonghao

3. customer (客户表)

customer (id, customerming, xingming, mima, xingbie, nianling, shoujihao, addtime)

主键 PK: id

候选键 CK: customerming

4. hotel_profile (酒店简介表)

hotel_profile (id, jiudianmingcheng, leibie, xingji, jiudiantupian, jiudiandizhi, fuwurexian, jiudianjieshao, addtime)

主键 PK: id

5. hotel_review (酒店评论表)

hotel_review (id, refid, userid, nickname, content, reply, score, addtime)

主键 PK: id

6. room_review (客房评论表)

room_review (id, refid, userid, nickname, content, reply, score, addtime)

主键 PK: id

7. favorite (收藏表)

favorite (id, userid, refid, tablename, name, picture, addtime)

主键 PK: id

8. notice (公告表)

notice (id, title, content, picture, addtime)

主键 PK: id

9. system_config (系统配置表)

system_config (id, name, value)

主键 PK: id

10. operation_log (操作日志表)

operation_log (id, operator_type, operator_id, business_type, action, remark, created_at)

主键 PK: id

11. auth_token (认证令牌表)

auth_token (id, userid, username, tablename, role, auth_token, addtime, expiredtime)

主键 PK: id

12. employee_performance (员工绩效表)

employee_performance (id, employee_id, assessor_id, period_start, period_end, checkin_count, bill_count, payment_count, cleaning_count, maintenance_count, operation_count, manager_score, total_score, grade, remark, created_at)

主键 PK: id

候选键 CK: employee_id, period_start, period_end

外键 FK: employee_id 引用 employee (id),

约束名 fk_employee_performance_employee, 级联更新与删除 (ON UPDATE CASCADE, ON DELETE CASCADE)

外键 FK: assessor_id 引用 employee (id),

约束名 fk_employee_performance_assessor, 级联更新、删除置空 (ON UPDATE CASCADE, ON DELETE SET NULL)

5.2 模式优化

5.2.1 客房资源管理模块

1.如果直接把客房名称、房型等字段全部放进一张表，会造成明显的数据冗余。例如同一种房型下的多间房间会重复保存同样的客房名称、图片、介绍、设施和基础价格。一旦房型基础价格调整，就需要修改多行数据，容易产生更新异常。因此当前设计将其拆分为：

`'room_catalog'`：描述“卖什么房” `'room_instance'`：描述“具体哪一间房”

这样可以使客房目录信息只保存一次，实体房间通过外键引用目录主键

2.如果只在 `'room_instance'` 中保存当前状态，那么虽然能知道房间现在是否可用，但无法回答“为什么变成这个状态”“由谁操作”“何时发生变更”等问题。因此系统将房态控制拆为两层：

`'room_instance.status'` 保存当前房态 `'room_status_log'` 保存状态变更历史

3. 本模块主要关系模式均可认为至少达到第三范式。

关系模式	关键函数依赖	规范化说明
hotel_profile	id->酒店名称、类别、星级、地址...	非主属性完全依赖主键，无传递依赖
room_category	id->room_category, room_category->id	类型字典表，满足 3NF
room_catalog	id->客房目录描述信息	房型描述依赖目录主键，酒店与房型通过外键关联，避免把基础实体信息混入本表
room_instance	id->room_catalog_id, room_no, floor_no, status..., room_no->id	当前房态依赖实体房间主键，无部分依赖
price_policy	id->room_catalog_id, 起止日期, 价格倍率...	价格策略独立存放，避免重复写入客房表
room_status_log	id->room_id, old_status, new_status, source_type...	每条日志唯一表示一次变更，满足 3NF
booking_order	id->customer_id, room_catalog_id, expected_checkin_date...	主单负责订单头信息，不再承载多房间明细
booking_room	id->booking_id, room_id, room_price, stay_start_date...	房间明细只依赖本明细主键
housekeeping_task	id->room_id, task_type, task_status...	清扫任务独立管理，消除“房态=清扫中”场景中的附加描述冗余
maintenance_record	id->room_id, issue_desc, maintenance_status...	维修业务独立建表，消除维修描述与房态字段耦合

表 3 客房资源管理模块关系模式

5.2.2 预定与入住模块

(1) 第一范式 (1NF)

所有字段均为原子属性，不存在重复组。

例如：一个入住人对应一条入住记录；一个联系电话只保存一个号码。

因此满足第一范式。

(2) 第二范式 (2NF)

所有非主属性完全依赖于主键。

例如：booking_id → customer_id, room_id, booking_amount 所有字段均依赖于 booking_id。

因此满足第二范式。

(3) 第三范式 (3NF)

不存在非主属性对主键的传递依赖。

例如：客户联系电话仅保存在 customer 表；房态仅保存在 room_instance 表。

因此满足第三范式。

表名	主键	外键
customer	customer_id	无
booking_order	booking_id	customer_id、room_id
booking_status_log	log_id	booking_id
checkin_record	checkin_id	booking_id、room_id、employee_id
checkin_guest	guest_id	checkin_id

表 4 预定与入住主键与外键设计

5.2.3 财务结算与支付退款模块

1. bill 关系模式优化分析

函数依赖：

id → booking_id, checkin_id, customer_id, room_amount, service_amount, discount_amount, payable_amount, paid_amount, refund_amount, status

非主属性依赖分析：所有非主属性（如金额、状态等）都完全函数依赖于主键 id，不依赖于其他非主属性。

传递依赖检查：payable_amount 是由 room_amount + service_amount - discount_amount 计算得出，这是一种计算依赖，但在数据库设计中作为冗余字段存储以提高查询性能。

结论：此关系模式不存在部分或传递依赖，直接满足 3NF 和 BCNF。

2. payment_record 关系模式优化分析

函数依赖：

id → payment_no, bill_id, pay_method, pay_amount, pay_status, paid_at

payment_no → id, bill_id, pay_method, pay_amount, pay_status, paid_at （候选键）

分析：不包含部分依赖或传递依赖。

结论：满足 3NF 和 BCNF。

3. refund_record 关系模式优化分析

函数依赖:

id → payment_id, refund_no, refund_amount, refund_reason, refund_status

refund_no → id, payment_id, refund_amount, refund_reason, refund_status (候选键)

分析: 不包含部分依赖或传递依赖。

结论: 满足 3NF 和 BCNF。

5.2.4 后台管理模块

1. 所有表均有**唯一主键**，无部分依赖与传递依赖。
2. 酒店、客房、评价、订单、用户完全分离，消除冗余。
3. 使用**外键**建立关联，保证数据一致性。
4. 评价、收藏、日志统一设计，可扩展、易维护。
5. 所有字段原子性强，无复合属性。
6. 新增员工绩效表、登录令牌表，均满足第三范式，无冗余字段，主键唯一，外键关联对应主表。
7. 登录令牌通过 tablename 区分管理员 / 员工 / 用户，实现多角色统一鉴权。

5.3 表关系图

5.3.1 客房资源管理模块

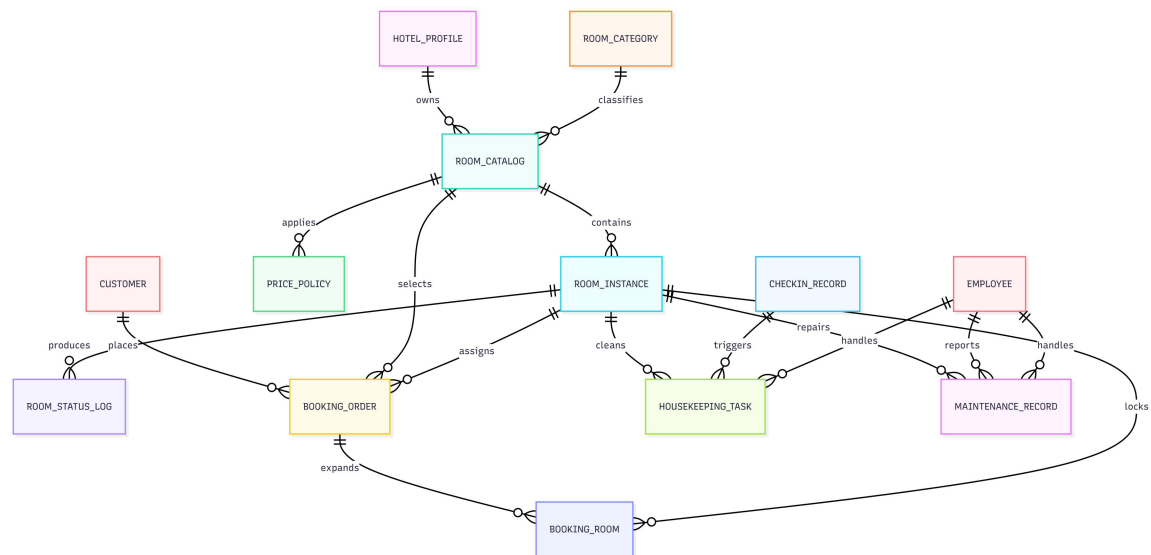


图 5 客房资源管理模块主要外键依赖关系

主要依赖关系：

1. room_catalog（客房目录表）的外键依赖：

room_catalog 通过 hotel_profile_id、room_category_id 关联酒店档案和客房类型。对应约束为 fk_room_catalog_hotel_profile、fk_room_catalog_room_category，删除规则均为 ON DELETE RESTRICT，防止已使用的酒店或房型被删除。

2. room_instance（实体房间表）的外键依赖：

room_instance 通过 room_catalog_id 关联 room_catalog。对应约束为 fk_room_instance_catalog，删除规则为 ON DELETE RESTRICT，保证已有实体房间时不能删除客房目录。

3. price_policy（房价策略表）的外键依赖：

price_policy 通过 room_catalog_id 关联 room_catalog。对应约束为 fk_price_policy_catalog，删除规则为 ON DELETE CASCADE，删除客房目录时同步删除价格策略。

4. room_status_log（房态变更日志表）的外键依赖：

room_status_log 通过 room_id 关联 room_instance。对应约束为 fk_room_status_log_room，删除规则为 ON DELETE CASCADE，删除实体房间时同步删除房态日志。

5. 客房业务使用关系:

booking_order 、 booking_room 、 checkin_record 、 housekeeping_task 、 maintenance_record 均通过 room_catalog_id 或 room_id 关联客房资源。其中订单表多采用 ON DELETE SET NULL 保留业务记录，入住、保洁、维修等直接占用房间的记录多采用 ON DELETE RESTRICT，防止删除正在被业务引用的实体房间。

5.3.2 入住与预定模块

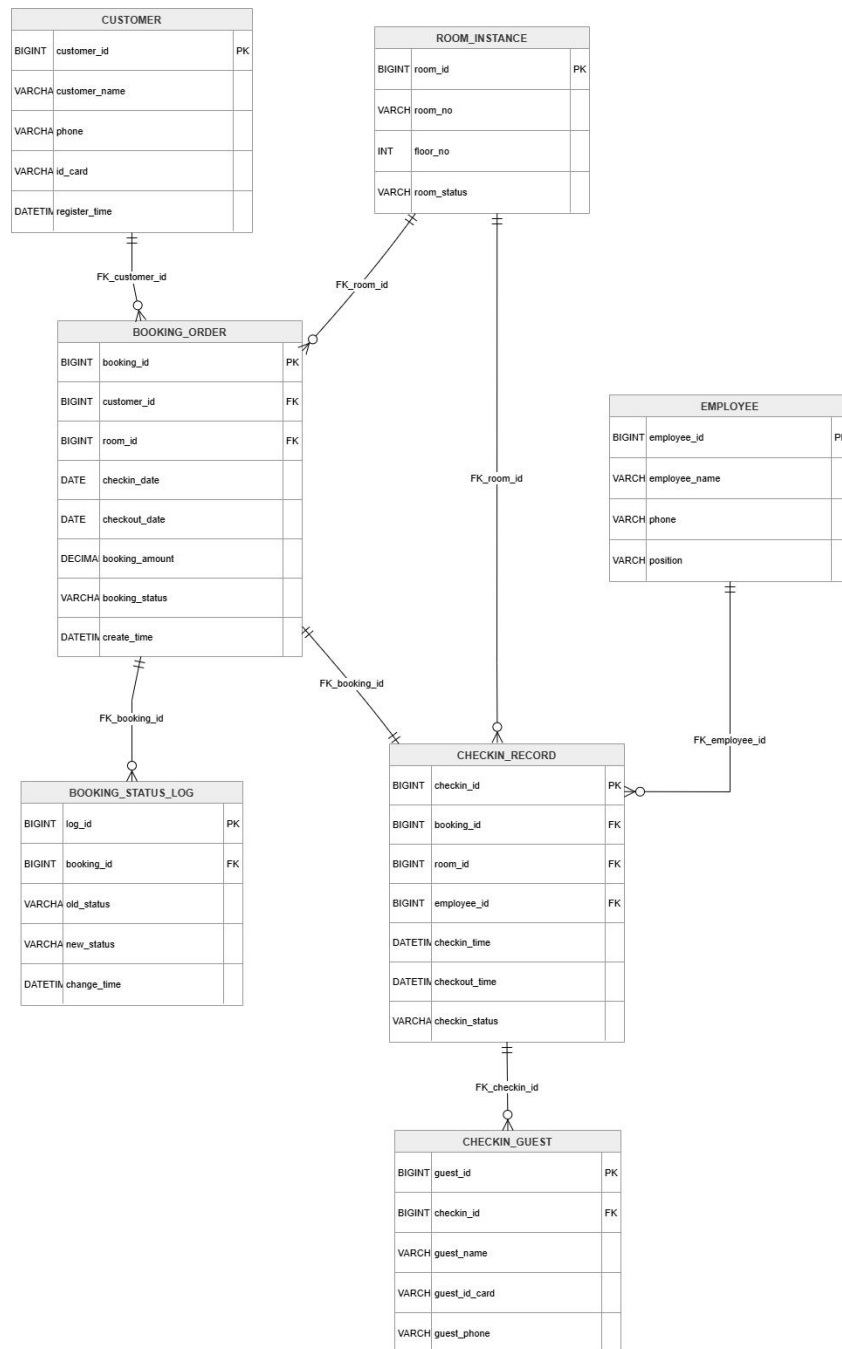


图 6 预定与入住模块 PDM

5.3.3 财务结算支付退款模块

1. bill (账单表) 的外键依赖:

bill 通过 booking_id 和 checkin_id 追溯业务来源。对应的物理约束为 fk_bill_booking 和 fk_bill_checkin, 删除规则均为 ON DELETE SET NULL。即删除预订订单或入住记录时, 账单记录保留, 外键字段置为 NULL, 保护财务数据不丢失。customer_id 在逻辑上关联客户, 但当前版本未建立物理外键约束, 由应用层维护一致性。

2. payment_record (支付记录表) 的外键依赖:

bill_id 通过约束 fk_payment_record_bill 引用 bill 表, 规则为 ON UPDATE CASCADE 和 ON DELETE CASCADE (删除账单时支付记录级联删除)。

3. refund_record (退款记录表) 的外键依赖:

payment_id 通过约束 fk_refund_record_payment 引用 payment_record 表, 同样采用级联更新和删除。

5.3.4 后台管理模块

4. 员工、管理员与操作日志

employee 与 operation_log: 一个员工可以产生多条操作日志, operation_log.operator_id 关联 employee.id, 用于绩效统计和操作追溯。

admin_user 与 operation_log: 一个管理员可以产生多条操作日志, operation_log.operator_id 关联 admin_user.id。

5. 评价相关依赖

hotel_profile 与 hotel_review: 一个酒店可以拥有多条酒店评价记录, hotel_review.refid 关联 hotel_profile.id。评价提交后通过触发器更新 hotel_profile 中的 review_count 和 avg_score。

room_catalog 与 room_review: 一个客房目录可以拥有多条客房评价记录, room_review.refid 关联 room_catalog.id。评价提交后通过触发器更新 room_catalog 中的 review_count 和 avg_score。

6. 收藏依赖

customer 与 favorite: 一个客户可以收藏多个酒店或客房对象, favorite.userid 关联 customer.id。

7. 公告与系统配置

notice 和 system_config 为独立辅助表, 不依赖其他业务主表, 由管理员单独维护。

8. 员工与员工表现

employee (1) —— 一对多 —— employee_performance (N)

一个员工可有多条绩效考核记录, employee_performance.employee_id 关联 employee.id; 管理员可作为考核人, assessor_id 关联 admin_user.id。

9. 登录令牌关联

admin_user/employee/customer (1) —— 一对多 —— auth_token (N)

管理员、员工、用户均可生成多条登录令牌, auth_token.userid 关联对应用户主键, tablename 区分用户类型。

6 表的设计实现

6.1 表结构数据字典描述

表名	中文含义	主要字段	主键/候选键	完整性约束及说明
system_confing	系统配置表	id, name, value	PK: id	保存首页轮播、图片等配置参数, name 为配置项名称。
admin_user	管理员表	id, username, password, role, addtime	PK: id	管理员账号信息, role 默认 admin。
customer	用户表	id, addtime, customerming, xingming, mima, xingbie, nianling, shoujihao	PK: id; UK: customerming	客户登录名唯一, 供预订、收藏、评价、账单等业务引用。
employee	员工表	id, addtime, employeegonghao, employeexingming, mima, xingbie, lianxidianhua, touxiang, ruzhishijian, zhiwei	PK: id; UK: employeegonghao	员工工号唯一, 供入住办理、保洁、维修、绩效统计和操作日志引用。
auth_token	认证令牌表	id, userid, username, tablename, role, auth_token, addtime, expiredtime	PK: id	tablename 区分管理员、员工和用户, 记录登录令牌及过期时间。
hotel_profile	酒店简介表	id, addtime, jiudianmingcheng, leibie, xingji, jiudiantupian, jiudiandizhi, fuwurexian, jiudianjieshao, review_count, avg_score	PK: id	酒店基础资料; 评论触发器维护 review_count 与 avg_score。



表名	中文含义	主要字段	主键/候选键	完整性约束及说明
hotel_review	酒店评论表	id, addtime, refid, userid, nickname, content, reply, score	PK: id	refid 逻辑关联 hotel_profile.id; 新增评论后触发器回写酒店评分。
room_category	客房类型表	id, addtime, room_category	PK: id; UK: room_category	房型字典表, 避免房型名称重复录入。
room_catalog	酒店客房目录表	id, addtime, kefangmingcheng, room_category, kefangtupian, kefangjiage, shuliang, jiudianmingcheng, jiudiandizhi, kefangsheshi, kefangjieshao, clicknum, hotel_profile_id, room_category_id, review_count, avg_score	PK: id; FK: hotel_profile_id, room_category_id	描述可销售房型; 外键关联酒店和房型, 评论触发器维护评分统计。
room_review	客房评论表	id, addtime, refid, userid, nickname, content, reply, score	PK: id	refid 逻辑关联 room_catalog.id; 新增评论后触发器回写客房评分。
room_instance	实体房间表	id, room_catalog_id, room_no, floor_no, status, created_at, updated_at	PK: id; UK: room_no; FK: room_catalog_id	记录每一间实体房间; status 包含 available、reserved、occupied、cleaning、maintenance。



表名	中文含义	主要字段	主键/候选键	完整性约束及说明
price_policy	价格策略表	id, room_catalog_id, policy_name, start_date, end_date, price_multiplier, priority, enabled	PK: id; FK: room_catalog_id	按日期区间和优先级维护房价倍率, 删除客房目录时级联删除策略。
booking_order	客房预订主单表	id, addtime, yudingbianhao, kefangmingcheng, room_category, kefangjiage, shuliang, zongjine, customer_id, room_catalog_id, expected_checkin_date, expected_checkout_date, booking_status, room_instance_id	PK: id; UK: yudingbianhao; FK: customer_id, room_catalog_id, room_instance_id	记录预订主信息; 状态变更写入日志, 取消订单时释放已锁定房间。
booking_room	订单房间明细表	id, booking_id, room_id, room_price, stay_start_date, stay_end_date, room_status, created_at	PK: id; UK: booking_id+room_id; FK: booking_id, room_id	解决一个订单预订多间实体房间的问题; 插入明细时触发器锁定房态。
booking_status_log	预订状态日志表	id, booking_id, old_status, new_status, operator_id, remark, created_at	PK: id; FK: booking_id	记录订单创建、入住、取消、完成等状态流转过程。
checkin_assignment	入住安排表	id, addtime, yudingbianhao, jiudianmingcheng, fangjianleixing, shuliang, customerming, xingming, shoujihao, fangjianhao, crossuserid, crossrefid	PK: id	保留原系统入住安排数据, 作为升级前业务数据兼容表。



表名	中文含义	主要字段	主键/候选键	完整性约束及说明
checkin_record	入住记录表	id, booking_id, room_id, customer_id, employee_id, checkin_time, checkout_time, status, remark	PK: id; FK: booking_id, room_id, customer_id, employee_id	办理入住时占用房间;退房时改为待清扫并生成清扫任务。
checkin_guest	入住人明细表	id, checkin_id, guest_name, id_card_no, phone, is_primary, created_at	PK: id; FK: checkin_id	一条入住记录可对应多个入住人,删除入住记录时级联删除明细。
bill	账单表	id, booking_id, checkin_id, customer_id, room_amount, service_amount, discount_amount, payable_amount, paid_amount, refund_amount, status, created_at, updated_at	PK: id; FK: booking_id, checkin_id	账单至少可追溯到预订或入住;支付、退款触发器维护 paid_amount、refund_amount 和 status。
payment_record	支付记录表	id, bill_id, payment_no, pay_method, pay_amount, pay_status, paid_at, remark	PK: id; UK: payment_no; FK: bill_id	记录每次支付流水;成功支付后触发器更新账单实付金额。
refund_record	退款记录表	id, payment_id, refund_no, refund_amount, refund_reason, refund_status, created_at	PK: id; UK: refund_no; FK: payment_id	记录退款流水;成功退款后触发器汇总退款金额并更新账单状态。
favorite	收藏表	id, addtime, userid, refid, tablename, name, picture, type, inteltype	PK: id	userid 逻辑关联客户, tablename 与 refid 指向被收藏的



表名	中文含义	主要字段	主键/候选键	完整性约束及说明
				酒店或客房
notice	酒店公告表	id, addtime, title, introduction, picture, content	PK: id	管理员维护公告内容和展示图片。
operation_log	操作日志表	id, operator_type, operator_id, business_type, business_id, action, remark, created_at	PK: id	统一记录系统、客户、员工、管理员的关键操作,支撑审计和绩效统计。
room_status_log	房态变更日志表	id, room_id, old_status, new_status, source_type, source_id, operator_id, remark, created_at	PK: id; FK: room_id	记录房态变化来源,可追溯预订、入住、保洁和维修导致的状态变更。
housekeeping_task	保洁任务表	id, room_id, checkin_id, assigned_employee_id, task_type, task_status, scheduled_time, completed_time, remark, created_at, updated_at	PK: id; FK: room_id, checkin_id, assigned_employee_id	退房、维修完成或人工创建保洁任务;完成后触发器释放房间为空闲。
maintenance_record	维修记录表	id, room_id, reported_by_employee_id, handled_by_employee_id, issue_desc, maintenance_status, start_time, end_time, cost_amount, remark	PK: id; FK: room_id, reported_by_employee_id, handled_by_employee_id	报修时触发器将房态置为维修;维修完成后转为待清扫并生成保洁任务。
employee_performance	员工绩效表	id, employee_id, assessor_id, period_start, period_end, checkin_count, bill_count,	PK: id; UK: employee_id+period_start+period_end; FK: employee_id, assessor_id	按考核周期记录员工业务量和评分结果,支持绩效统计。

表名	中文含义	主要字段	主键/候选键	完整性约束及说明
		payment_count, cleaning_count, maintenance_count, operation_count, manager_score, total_score, grade, remark, created_at		

表 5 表结构数据字典

其中，系统中所有业务主表均设置自增主键；客户登录名、员工工号、房间号、预订编号、支付流水号、退款流水号等具有业务唯一性的字段设置唯一约束；可被历史数据引用的业务表在删除规则上多采用 RESTRICT 或 SET NULL，以避免误删造成订单、账单和日志丢失。

复杂完整性主要通过触发器实现：预订或订单房间明细生成后锁定房态，取消预订释放房间；入住后房态置为 occupied，退房后置为 cleaning 并生成保洁任务；维修上报后置为 maintenance，维修完成后转入清扫；支付与退款触发器自动回写账单金额和状态；酒店、客房评论触发器维护评价数量和平均分。

6.2 表的创建

1. 基础表与优化后业务表创建脚本

```
CREATE TABLE IF NOT EXISTS `system_config` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `name` varchar(100) NOT NULL COMMENT '配置参数名称',
  `value` varchar(100) DEFAULT NULL COMMENT 'config value',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8mb4 COMMENT='配置文件';

CREATE TABLE IF NOT EXISTS `hotel_review` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `refid` bigint NOT NULL COMMENT '关联表 id',
  `userid` bigint NOT NULL COMMENT '用户 id',
  `nickname` varchar(200) DEFAULT NULL COMMENT 'username',
  `content` longtext NOT NULL COMMENT '评论内容',
  `reply` longtext COMMENT '回复内容',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1108 DEFAULT CHARSET=utf8mb4 COMMENT='酒店简介评论表';

CREATE TABLE IF NOT EXISTS `room_review` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
```



```

`refid` bigint NOT NULL COMMENT '关联表 id',
`userid` bigint NOT NULL COMMENT '用户 id',
`nickname` varchar(200) DEFAULT NULL COMMENT 'username',
`content` longtext NOT NULL COMMENT '评论内容',
`reply` longtext COMMENT '回复内容',
PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1208 DEFAULT CHARSET=utf8mb4 COMMENT='legacy table';

CREATE TABLE IF NOT EXISTS `hotel_profile` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `jiudianmingcheng` varchar(200) NOT NULL COMMENT '酒店名称',
  `leibie` varchar(200) NOT NULL COMMENT '类别',
  `xingji` varchar(200) NOT NULL COMMENT '星级',
  `jiudiantupian` varchar(200) DEFAULT NULL COMMENT '酒店图片',
  `jiudiandizhi` varchar(200) DEFAULT NULL COMMENT '酒店地址',
  `fuwurexian` varchar(200) DEFAULT NULL COMMENT '服务热线',
  `jiudianjieshao` longtext COMMENT '酒店介绍',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1002 DEFAULT CHARSET=utf8mb4 COMMENT='legacy table';

CREATE TABLE IF NOT EXISTS `room_catalog` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `kefangmingcheng` varchar(200) NOT NULL COMMENT '客房名称',
  `room_category` varchar(200) NOT NULL COMMENT '客房类型',
  `kefangtupian` varchar(200) NOT NULL COMMENT '客房图片',
  `kefangjiage` float NOT NULL COMMENT '客房价格',
  `shuliang` int DEFAULT NULL COMMENT '数量',
  `jiudianmingcheng` varchar(200) DEFAULT NULL COMMENT '酒店名称',
  `jiudiandizhi` varchar(200) DEFAULT NULL COMMENT '酒店地址',
  `kefangsheshi` longtext COMMENT '客房设施',
  `kefangjieshao` longtext COMMENT '客房介绍',
  `clicknum` int DEFAULT '0' COMMENT '点击次数',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=3008 DEFAULT CHARSET=utf8mb4 COMMENT='酒店客房';

CREATE TABLE IF NOT EXISTS `room_category` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `room_category` varchar(200) NOT NULL COMMENT '客房类型',
  PRIMARY KEY (`id`),
  UNIQUE KEY `room_category` (`room_category`)
) ENGINE=InnoDB AUTO_INCREMENT=2008 DEFAULT CHARSET=utf8mb4 COMMENT='客房类型';

```



```
CREATE TABLE IF NOT EXISTS `booking_order` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',  
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `yudingbianhao` varchar(200) DEFAULT NULL COMMENT '预定编号',  
  `kefangmingcheng` varchar(200) NOT NULL COMMENT '客房名称',  
  `room_category` varchar(200) NOT NULL COMMENT '客房类型',  
  `kefangjiage` float NOT NULL COMMENT '客房价格',  
  `shuliang` int DEFAULT NULL COMMENT '数量',  
  `zongjine` float DEFAULT NULL COMMENT 'total amount',  
  `kefangtupian` varchar(200) DEFAULT NULL COMMENT '客房图片',  
  `jiudianmingcheng` varchar(200) DEFAULT NULL COMMENT '酒店名称',  
  `jiudiandizhi` varchar(200) DEFAULT NULL COMMENT '酒店地址',  
  `customerming` varchar(200) DEFAULT NULL COMMENT 'username',  
  `xingming` varchar(200) DEFAULT NULL COMMENT '姓名',  
  `shoujihao` varchar(200) DEFAULT NULL COMMENT 'mobile phone',  
  `yudingriqi` date DEFAULT NULL COMMENT '预定日期',  
  `ispay` varchar(200) DEFAULT 'unpaid' COMMENT 'payment status',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `yudingbianhao` (`yudingbianhao`)  
) ENGINE=InnoDB AUTO_INCREMENT=6008 DEFAULT CHARSET=utf8mb4 COMMENT='客房预定';  
  
CREATE TABLE IF NOT EXISTS `notice` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',  
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `title` varchar(200) NOT NULL COMMENT '标题',  
  `introduction` longtext COMMENT 'introduction',  
  `picture` varchar(200) NOT NULL COMMENT '图片',  
  `content` longtext NOT NULL COMMENT '内容',  
  PRIMARY KEY (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=93 DEFAULT CHARSET=utf8mb4 COMMENT='酒店公告';  
  
CREATE TABLE IF NOT EXISTS `checkin_assignment` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',  
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `yudingbianhao` varchar(200) DEFAULT NULL COMMENT '预定编号',  
  `jiudianmingcheng` varchar(200) NOT NULL COMMENT '酒店名称',  
  `fangjianleixing` varchar(200) NOT NULL COMMENT '房间类型',  
  `shuliang` float DEFAULT NULL COMMENT '数量',  
  `kefangtupian` varchar(200) DEFAULT NULL COMMENT '客房图片',  
  `customerming` varchar(200) DEFAULT NULL COMMENT 'username',  
  `xingming` varchar(200) DEFAULT NULL COMMENT '姓名',  
  `shoujihao` varchar(200) DEFAULT NULL COMMENT 'mobile phone',  
  `fangjianhao` varchar(200) DEFAULT NULL COMMENT 'room number',
```



```
`crossuserid` bigint DEFAULT NULL COMMENT '跨表用户 id',
`crossrefid` bigint DEFAULT NULL COMMENT '跨表主键 id',
PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=78 DEFAULT CHARSET=utf8mb4 COMMENT='入住安排';

CREATE TABLE IF NOT EXISTS `favorite` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `userid` bigint NOT NULL COMMENT '用户 id',
  `refid` bigint DEFAULT NULL COMMENT '收藏 id',
  `tablename` varchar(200) DEFAULT NULL COMMENT '表名',
  `name` varchar(200) NOT NULL COMMENT '收藏名称',
  `picture` varchar(200) NOT NULL COMMENT '收藏图片',
  `type` varchar(200) DEFAULT '1' COMMENT 'favorite type',
  `inteltype` varchar(200) DEFAULT NULL COMMENT '推荐类型',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1305 DEFAULT CHARSET=utf8mb4 COMMENT='legacy table';

CREATE TABLE IF NOT EXISTS `auth_token` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `userid` bigint NOT NULL COMMENT '用户 id',
  `username` varchar(100) NOT NULL COMMENT 'username',
  `tablename` varchar(100) DEFAULT NULL COMMENT '表名',
  `role` varchar(100) DEFAULT NULL COMMENT '角色',
  `auth_token` varchar(200) NOT NULL COMMENT '密码',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '新增时间',
  `expiredtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '过期时间',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=1406 DEFAULT CHARSET=utf8mb4 COMMENT='legacy table';

CREATE TABLE IF NOT EXISTS `admin_user` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `username` varchar(100) NOT NULL COMMENT 'username',
  `password` varchar(100) NOT NULL COMMENT '密码',
  `role` varchar(100) DEFAULT 'admin' COMMENT 'role',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '新增时间',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COMMENT='legacy table';

CREATE TABLE IF NOT EXISTS `customer` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `customerming` varchar(200) NOT NULL COMMENT 'username',
  `xingming` varchar(200) NOT NULL COMMENT '姓名',
```




```

`mima` varchar(200) NOT NULL COMMENT '密码',
`xingbie` varchar(200) DEFAULT NULL COMMENT '性别',
`nianling` int DEFAULT NULL COMMENT '年龄',
`shoujihao` varchar(200) DEFAULT NULL COMMENT 'mobile phone',
PRIMARY KEY (`id`),
UNIQUE KEY `customerming` (`customerming`)
) ENGINE=InnoDB AUTO_INCREMENT=4008 DEFAULT CHARSET=utf8mb4 COMMENT='用户';

CREATE TABLE IF NOT EXISTS `employee` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '主键',
  `addtime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `employeegonghao` varchar(200) DEFAULT NULL COMMENT '员工工号',
  `employeexingming` varchar(200) DEFAULT NULL COMMENT '员工姓名',
  `mima` varchar(200) DEFAULT NULL COMMENT '密码',
  `xingbie` varchar(200) DEFAULT NULL COMMENT '性别',
  `lianxidiahua` varchar(200) DEFAULT NULL COMMENT '联系电话',
  `touxiang` varchar(200) DEFAULT NULL COMMENT '头像',
  `ruzhihishijian` date DEFAULT NULL COMMENT '入职时间',
  `zhiwei` varchar(200) DEFAULT NULL COMMENT '职位',
  PRIMARY KEY (`id`),
  UNIQUE KEY `employeegonghao` (`employeegonghao`)
) ENGINE=InnoDB AUTO_INCREMENT=5008 DEFAULT CHARSET=utf8mb4 COMMENT='员工';

CREATE TABLE `room_instance` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `room_catalog_id` bigint NOT NULL COMMENT 'room catalog id',
  `room_no` varchar(50) NOT NULL COMMENT 'room number',
  `floor_no` int NOT NULL DEFAULT 1 COMMENT 'floor number',
  `status` varchar(30) NOT NULL DEFAULT 'available' COMMENT 'available, reserved, occupied, cleaning, maintenance',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT 'updated time',
  PRIMARY KEY (`id`), UNIQUE KEY `uk_room_instance_no` (`room_no`),
  KEY `idx_room_instance_catalog` (`room_catalog_id`), KEY `idx_room_instance_status` (`status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='specific hotel room instances';

CREATE TABLE `price_policy` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `room_catalog_id` bigint NOT NULL COMMENT 'room catalog id',
  `policy_name` varchar(100) NOT NULL COMMENT 'policy name',
  `start_date` date NOT NULL COMMENT 'start date',
  `end_date` date NOT NULL COMMENT 'end date',

```

```
`price_multiplier` decimal(6,2) NOT NULL DEFAULT 1.00 COMMENT 'price multiplier',
`priority` int NOT NULL DEFAULT 1 COMMENT 'priority',
`enabled` tinyint NOT NULL DEFAULT 1 COMMENT 'enabled flag',
PRIMARY KEY (`id`), KEY `idx_price_policy_room_date` (`room_catalog_id`, `start_date`,
`end_date`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='room price policy';

CREATE TABLE `operation_log` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `operator_type` varchar(30) NOT NULL DEFAULT 'system' COMMENT 'admin, employee, customer, system',
  `operator_id` bigint DEFAULT NULL COMMENT 'operator id',
  `business_type` varchar(50) NOT NULL COMMENT 'business type',
  `business_id` bigint DEFAULT NULL COMMENT 'business id',
  `action` varchar(100) NOT NULL COMMENT 'operation action',
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
  PRIMARY KEY (`id`), KEY `idx_operation_business` (`business_type`, `business_id`),
  KEY `idx_operation_operator` (`operator_type`, `operator_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='operation audit log';

CREATE TABLE `room_status_log` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `room_id` bigint NOT NULL COMMENT 'room instance id',
  `old_status` varchar(30) DEFAULT NULL COMMENT 'old status',
  `new_status` varchar(30) NOT NULL COMMENT 'new status',
  `source_type` varchar(50) NOT NULL COMMENT 'source business type',
  `source_id` bigint DEFAULT NULL COMMENT 'source business id',
  `operator_id` bigint DEFAULT NULL COMMENT 'operator id',
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
  PRIMARY KEY (`id`), KEY `idx_room_status_room_time` (`room_id`, `created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='room status change log';

CREATE TABLE `booking_status_log` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `booking_id` bigint NOT NULL COMMENT 'booking order id',
  `old_status` varchar(30) DEFAULT NULL COMMENT 'old status',
  `new_status` varchar(30) NOT NULL COMMENT 'new status',
  `operator_id` bigint DEFAULT NULL COMMENT 'operator id',
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
  PRIMARY KEY (`id`), KEY `idx_booking_status_booking_time` (`booking_id`, `created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='booking status change log';
```



```
CREATE TABLE `booking_room` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',  
  `booking_id` bigint NOT NULL COMMENT 'booking order id',  
  `room_id` bigint NOT NULL COMMENT 'room instance id',  
  `room_price` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'locked room price',  
  `stay_start_date` date NOT NULL COMMENT 'planned checkin date',  
  `stay_end_date` date NOT NULL COMMENT 'planned checkout date',  
  `room_status` varchar(30) NOT NULL DEFAULT 'reserved' COMMENT 'reserved, checked_in, cancelled, completed',  
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_booking_room` (`booking_id`, `room_id`),  
  KEY `idx_booking_room_room_date` (`room_id`, `stay_start_date`, `stay_end_date`),  
  KEY `idx_booking_room_status` (`room_status`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='booking room detail for one order with multiple rooms';  
  
CREATE TABLE `checkin_record` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',  
  `booking_id` bigint DEFAULT NULL COMMENT 'booking order id',  
  `room_id` bigint NOT NULL COMMENT 'room instance id',  
  `customer_id` bigint DEFAULT NULL COMMENT 'customer id',  
  `employee_id` bigint DEFAULT NULL COMMENT 'employee id',  
  `checkin_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'checkin time',  
  `checkout_time` datetime DEFAULT NULL COMMENT 'checkout time',  
  `status` varchar(30) NOT NULL DEFAULT 'checked_in' COMMENT 'checked_in, checked_out, cancelled',  
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',  
  PRIMARY KEY (`id`), KEY `idx_checkin_booking` (`booking_id`),  
  KEY `idx_checkin_room` (`room_id`), KEY `idx_checkin_customer` (`customer_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='checkin master record';  
  
CREATE TABLE `checkin_guest` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',  
  `checkin_id` bigint NOT NULL COMMENT 'checkin record id',  
  `guest_name` varchar(100) NOT NULL COMMENT 'guest name',  
  `id_card_no` varchar(30) DEFAULT NULL COMMENT 'id card number',  
  `phone` varchar(30) DEFAULT NULL COMMENT 'phone number',  
  `is_primary` tinyint NOT NULL DEFAULT 0 COMMENT 'primary guest flag',  
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',  
  PRIMARY KEY (`id`), KEY `idx_checkin_guest_checkin` (`checkin_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='checkin guest detail';
```



```
CREATE TABLE `bill` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',  
  `booking_id` bigint DEFAULT NULL COMMENT 'booking order id',  
  `checkin_id` bigint DEFAULT NULL COMMENT 'checkin record id',  
  `customer_id` bigint DEFAULT NULL COMMENT 'customer id',  
  `room_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'room amount',  
  `service_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'service amount',  
  `discount_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'discount amount',  
  `payable_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'payable amount',  
  `paid_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'paid amount',  
  `refund_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'refund amount',  
  `status` varchar(30) NOT NULL DEFAULT 'unpaid' COMMENT 'unpaid, partial, paid, refunded',  
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',  
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT 'updated time',  
  PRIMARY KEY (`id`), KEY `idx_bill_booking` (`booking_id`),  
  KEY `idx_bill_checkin` (`checkin_id`), KEY `idx_bill_customer` (`customer_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='bill master';  
  
CREATE TABLE `payment_record` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',  
  `bill_id` bigint NOT NULL COMMENT 'bill id',  
  `payment_no` varchar(80) NOT NULL COMMENT 'payment number',  
  `pay_method` varchar(30) NOT NULL DEFAULT 'cash' COMMENT 'cash, card, alipay, wechat',  
  `pay_amount` decimal(10,2) NOT NULL COMMENT 'pay amount',  
  `pay_status` varchar(30) NOT NULL DEFAULT 'success' COMMENT 'success, failed, cancelled',  
  `paid_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'paid time',  
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',  
  PRIMARY KEY (`id`), UNIQUE KEY `uk_payment_no` (`payment_no`), KEY `idx_payment_bill`  
  (`bill_id`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='payment record';  
  
CREATE TABLE `refund_record` (  
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',  
  `payment_id` bigint NOT NULL COMMENT 'payment id',  
  `refund_no` varchar(80) NOT NULL COMMENT 'refund number',  
  `refund_amount` decimal(10,2) NOT NULL COMMENT 'refund amount',  
  `refund_reason` varchar(500) DEFAULT NULL COMMENT 'refund reason',  
  `refund_status` varchar(30) NOT NULL DEFAULT 'success' COMMENT 'success, failed, cancelled',  
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
```



```

PRIMARY KEY (`id`), UNIQUE KEY `uk_refund_no` (`refund_no`), KEY `idx_refund_payment`
(`payment_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='refund record';

CREATE TABLE `housekeeping_task` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `room_id` bigint NOT NULL COMMENT 'room instance id',
  `checkin_id` bigint DEFAULT NULL COMMENT 'checkout source checkin id',
  `assigned_employee_id` bigint DEFAULT NULL COMMENT 'assigned housekeeping employee id',
  `task_type` varchar(30) NOT NULL DEFAULT 'checkout_cleaning' COMMENT 'daily_cleaning,
checkout_cleaning, deep_cleaning',
  `task_status` varchar(30) NOT NULL DEFAULT 'pending' COMMENT 'pending, assigned, in_pr
ogress, completed, cancelled',
  `scheduled_time` datetime DEFAULT NULL COMMENT 'scheduled time',
  `completed_time` datetime DEFAULT NULL COMMENT 'completed time',
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP C
OMMENT 'updated time',
  PRIMARY KEY (`id`),
  KEY `idx_housekeeping_room_status` (`room_id`, `task_status`),
  KEY `idx_housekeeping_employee` (`assigned_employee_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='room housekeeping task';

CREATE TABLE `maintenance_record` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
  `room_id` bigint NOT NULL COMMENT 'room instance id',
  `reported_by_employee_id` bigint DEFAULT NULL COMMENT 'report employee id',
  `handled_by_employee_id` bigint DEFAULT NULL COMMENT 'handle employee id',
  `issue_desc` varchar(500) NOT NULL COMMENT 'maintenance issue description',
  `maintenance_status` varchar(30) NOT NULL DEFAULT 'reported' COMMENT 'reported, proces
sing, completed, cancelled',
  `start_time` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'reported time',
  `end_time` datetime DEFAULT NULL COMMENT 'finished time',
  `cost_amount` decimal(10,2) NOT NULL DEFAULT 0.00 COMMENT 'maintenance cost',
  `remark` varchar(500) DEFAULT NULL COMMENT 'remark',
  PRIMARY KEY (`id`),
  KEY `idx_maintenance_room_status` (`room_id`, `maintenance_status`),
  KEY `idx_maintenance_reporter` (`reported_by_employee_id`),
  KEY `idx_maintenance_handler` (`handled_by_employee_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='room maintenance record';

CREATE TABLE `employee_performance` (

```

```

`id` bigint NOT NULL AUTO_INCREMENT COMMENT 'primary key',
`employee_id` bigint NOT NULL COMMENT 'employee id',
`assessor_id` bigint DEFAULT NULL COMMENT 'assessor employee id',
`period_start` date NOT NULL COMMENT 'assessment period start',
`period_end` date NOT NULL COMMENT 'assessment period end',
`checkin_count` int NOT NULL DEFAULT 0 COMMENT 'handled checkin count',
`bill_count` int NOT NULL DEFAULT 0 COMMENT 'generated bill count',
`payment_count` int NOT NULL DEFAULT 0 COMMENT 'handled payment count',
`cleaning_count` int NOT NULL DEFAULT 0 COMMENT 'completed cleaning task count',
`maintenance_count` int NOT NULL DEFAULT 0 COMMENT 'completed maintenance count',
`operation_count` int NOT NULL DEFAULT 0 COMMENT 'operation log count',
`manager_score` decimal(5,2) NOT NULL DEFAULT 0.00 COMMENT 'manager score',
`total_score` decimal(6,2) NOT NULL DEFAULT 0.00 COMMENT 'calculated total score',
`grade` varchar(20) NOT NULL DEFAULT 'C' COMMENT 'A, B, C, D',
`remark` varchar(500) DEFAULT NULL COMMENT 'remark',
`created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'created time',
PRIMARY KEY (`id`),
UNIQUE KEY `uk_employee_performance_period` (`employee_id`, `period_start`, `period_end`),
KEY `idx_employee_performance_assessor` (`assessor_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='employee assessment result';

```

2. 补充字段、索引与外键约束脚本

```

SET FOREIGN_KEY_CHECKS = 0;

ALTER TABLE `room_catalog`
  ADD COLUMN `hotel_profile_id` bigint DEFAULT NULL COMMENT 'hotel profile id',
  ADD COLUMN `room_category_id` bigint DEFAULT NULL COMMENT 'room category id',
  ADD COLUMN `review_count` int NOT NULL DEFAULT 0 COMMENT 'review count',
  ADD COLUMN `avg_score` decimal(4,2) NOT NULL DEFAULT 0.00 COMMENT 'average score';

ALTER TABLE `hotel_profile`
  ADD COLUMN `review_count` int NOT NULL DEFAULT 0 COMMENT 'review count',
  ADD COLUMN `avg_score` decimal(4,2) NOT NULL DEFAULT 0.00 COMMENT 'average score';

ALTER TABLE `room_review` ADD COLUMN `score` tinyint NOT NULL DEFAULT 5 COMMENT 'score from 1 to 5';

ALTER TABLE `hotel_review` ADD COLUMN `score` tinyint NOT NULL DEFAULT 5 COMMENT 'score from 1 to 5';

ALTER TABLE `booking_order`
  ADD COLUMN `customer_id` bigint DEFAULT NULL COMMENT 'customer id',
  ADD COLUMN `room_catalog_id` bigint DEFAULT NULL COMMENT 'room catalog id',
  ADD COLUMN `expected_checkin_date` date DEFAULT NULL COMMENT 'expected checkin date',

```



```
ADD COLUMN `expected_checkout_date` date DEFAULT NULL COMMENT 'expected checkout date'
,
ADD COLUMN `booking_status` varchar(30) NOT NULL DEFAULT 'created' COMMENT 'created, checked_in, cancelled, completed',
ADD COLUMN `room_instance_id` bigint DEFAULT NULL COMMENT 'assigned room id';

ALTER TABLE `room_catalog`
ADD KEY `idx_room_catalog_hotel` (`hotel_profile_id`),
ADD KEY `idx_room_catalog_category` (`room_category_id`);

ALTER TABLE `booking_order`
ADD KEY `idx_booking_order_customer` (`customer_id`),
ADD KEY `idx_booking_order_catalog` (`room_catalog_id`),
ADD KEY `idx_booking_order_dates` (`expected_checkin_date`, `expected_checkout_date`);

ALTER TABLE `room_catalog`
ADD CONSTRAINT `fk_room_catalog_hotel_profile`
FOREIGN KEY (`hotel_profile_id`) REFERENCES `hotel_profile` (`id`)
ON UPDATE CASCADE ON DELETE RESTRICT,
ADD CONSTRAINT `fk_room_catalog_room_category`
FOREIGN KEY (`room_category_id`) REFERENCES `room_category` (`id`)
ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE `room_instance`
ADD CONSTRAINT `fk_room_instance_catalog`
FOREIGN KEY (`room_catalog_id`) REFERENCES `room_catalog` (`id`)
ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE `price_policy`
ADD CONSTRAINT `fk_price_policy_catalog`
FOREIGN KEY (`room_catalog_id`) REFERENCES `room_catalog` (`id`)
ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE `room_status_log`
ADD CONSTRAINT `fk_room_status_log_room`
FOREIGN KEY (`room_id`) REFERENCES `room_instance` (`id`)
ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE `booking_order`
ADD CONSTRAINT `fk_booking_order_customer`
FOREIGN KEY (`customer_id`) REFERENCES `customer` (`id`)
ON UPDATE CASCADE ON DELETE SET NULL,
ADD CONSTRAINT `fk_booking_order_catalog`
FOREIGN KEY (`room_catalog_id`) REFERENCES `room_catalog` (`id`)
```



```
        ON UPDATE CASCADE ON DELETE SET NULL,
ADD CONSTRAINT `fk_booking_order_room_instance`
    FOREIGN KEY (`room_instance_id`) REFERENCES `room_instance` (`id`)
    ON UPDATE CASCADE ON DELETE SET NULL;

ALTER TABLE `booking_status_log`
    ADD CONSTRAINT `fk_booking_status_log_booking`
        FOREIGN KEY (`booking_id`) REFERENCES `booking_order` (`id`)
        ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE `booking_room`
    ADD CONSTRAINT `fk_booking_room_booking`
        FOREIGN KEY (`booking_id`) REFERENCES `booking_order` (`id`)
        ON UPDATE CASCADE ON DELETE CASCADE,
    ADD CONSTRAINT `fk_booking_room_room`
        FOREIGN KEY (`room_id`) REFERENCES `room_instance` (`id`)
        ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE `checkin_record`
    ADD CONSTRAINT `fk_checkin_record_booking`
        FOREIGN KEY (`booking_id`) REFERENCES `booking_order` (`id`)
        ON UPDATE CASCADE ON DELETE SET NULL,
    ADD CONSTRAINT `fk_checkin_record_room`
        FOREIGN KEY (`room_id`) REFERENCES `room_instance` (`id`)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    ADD CONSTRAINT `fk_checkin_record_customer`
        FOREIGN KEY (`customer_id`) REFERENCES `customer` (`id`)
        ON UPDATE CASCADE ON DELETE SET NULL,
    ADD CONSTRAINT `fk_checkin_record_employee`
        FOREIGN KEY (`employee_id`) REFERENCES `employee` (`id`)
        ON UPDATE CASCADE ON DELETE SET NULL;

ALTER TABLE `checkin_guest`
    ADD CONSTRAINT `fk_checkin_guest_record`
        FOREIGN KEY (`checkin_id`) REFERENCES `checkin_record` (`id`)
        ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE `bill`
    ADD CONSTRAINT `fk_bill_booking`
        FOREIGN KEY (`booking_id`) REFERENCES `booking_order` (`id`)
        ON UPDATE CASCADE ON DELETE SET NULL,
    ADD CONSTRAINT `fk_bill_checkin`
        FOREIGN KEY (`checkin_id`) REFERENCES `checkin_record` (`id`)
        ON UPDATE CASCADE ON DELETE SET NULL;
```



```
ALTER TABLE `payment_record`
  ADD CONSTRAINT `fk_payment_record_bill`
    FOREIGN KEY (`bill_id`) REFERENCES `bill` (`id`)
    ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE `refund_record`
  ADD CONSTRAINT `fk_refund_record_payment`
    FOREIGN KEY (`payment_id`) REFERENCES `payment_record` (`id`)
    ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE `housekeeping_task`
  ADD CONSTRAINT `fk_housekeeping_task_room`
    FOREIGN KEY (`room_id`) REFERENCES `room_instance` (`id`)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  ADD CONSTRAINT `fk_housekeeping_task_checkin`
    FOREIGN KEY (`checkin_id`) REFERENCES `checkin_record` (`id`)
    ON UPDATE CASCADE ON DELETE SET NULL,
  ADD CONSTRAINT `fk_housekeeping_task_employee`
    FOREIGN KEY (`assigned_employee_id`) REFERENCES `employee` (`id`)
    ON UPDATE CASCADE ON DELETE SET NULL;

ALTER TABLE `maintenance_record`
  ADD CONSTRAINT `fk_maintenance_record_room`
    FOREIGN KEY (`room_id`) REFERENCES `room_instance` (`id`)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  ADD CONSTRAINT `fk_maintenance_record_reporter`
    FOREIGN KEY (`reported_by_employee_id`) REFERENCES `employee` (`id`)
    ON UPDATE CASCADE ON DELETE SET NULL,
  ADD CONSTRAINT `fk_maintenance_record_handler`
    FOREIGN KEY (`handled_by_employee_id`) REFERENCES `employee` (`id`)
    ON UPDATE CASCADE ON DELETE SET NULL;

ALTER TABLE `employee_performance`
  ADD CONSTRAINT `fk_employee_performance_employee`
    FOREIGN KEY (`employee_id`) REFERENCES `employee` (`id`)
    ON UPDATE CASCADE ON DELETE CASCADE,
  ADD CONSTRAINT `fk_employee_performance_assessor`
    FOREIGN KEY (`assessor_id`) REFERENCES `employee` (`id`)
    ON UPDATE CASCADE ON DELETE SET NULL;
SET FOREIGN_KEY_CHECKS = 1;
```



3. 复杂数据完整性触发器脚本

```
DELIMITER $$

CREATE TRIGGER `trg_booking_after_insert_lock_room`
AFTER INSERT ON `booking_order`
FOR EACH ROW
BEGIN
    DECLARE v_old_status varchar(30);
    IF NEW.room_instance_id IS NOT NULL THEN
        SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_instance_id;
        IF v_old_status='available' THEN
            UPDATE `room_instance` SET status='reserved',updated_at=NOW() WHERE id=NEW.room_instance_id;
            INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_id,operator_id,remark)
            VALUES(NEW.room_instance_id,v_old_status,'reserved','booking_order',NEW.id,NULL,CONCAT('booking created: ',IFNULL(NEW.yudingbianhao,NEW.id)));
            END IF;
        END IF;
        INSERT INTO `booking_status_log`(booking_id,old_status,new_status,operator_id,remark)
        VALUES(NEW.id,NULL,NEW.booking_status,NULL,'booking inserted');
        INSERT INTO `operation_log`(operator_type,operator_id,business_type,business_id,action,remark)
        VALUES('system',NULL,'booking_order',NEW.id,'after_insert_booking',NEW.yudingbianhao);
    ;
END$$

CREATE TRIGGER `trg_booking_after_update_release_room`
AFTER UPDATE ON `booking_order`
FOR EACH ROW
BEGIN
    DECLARE v_current_status varchar(30);
    IF OLD.booking_status<>NEW.booking_status THEN
        INSERT INTO `booking_status_log`(booking_id,old_status,new_status,operator_id,remark)
        VALUES(NEW.id,OLD.booking_status,NEW.booking_status,NULL,'booking status changed');
        END IF;
        IF NEW.booking_status='cancelled' AND OLD.booking_status<>'cancelled' AND NEW.room_instance_id IS NOT NULL THEN
            SELECT status INTO v_current_status FROM `room_instance` WHERE id=NEW.room_instance_id;
```



```

        UPDATE `room_instance` SET status='available',updated_at=NOW() WHERE id=NEW.room
_instance_id AND status IN ('reserved','available');

        INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_i
d,operator_id,remark)

        VALUES(NEW.room_instance_id,v_current_status,'available','booking_order',NEW.id,
NULL,'booking cancelled and room released');

    END IF;

    IF NEW.booking_status='cancelled' AND OLD.booking_status<>'cancelled' THEN

        UPDATE `booking_room` SET room_status='cancelled' WHERE booking_id=NEW.id AND ro
om_status IN ('reserved','checked_in');

        INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_i
d,operator_id,remark)

        SELECT br.room_id, ri.status, 'available', 'booking_room', br.id, NULL, 'booking
cancelled and detail room released'

        FROM `booking_room` br JOIN `room_instance` ri ON ri.id=br.room_id

        WHERE br.booking_id=NEW.id AND ri.status='reserved';

        UPDATE `room_instance` ri JOIN `booking_room` br ON br.room_id=ri.id

        SET ri.status='available', ri.updated_at=NOW()

        WHERE br.booking_id=NEW.id AND ri.status='reserved';

    END IF;

END$$

CREATE TRIGGER `trg_booking_room_after_insert_lock_room`
AFTER INSERT ON `booking_room`
FOR EACH ROW
BEGIN
    DECLARE v_old_status varchar(30);

    IF NEW.room_status IN ('reserved','checked_in') THEN

        SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_id;

        IF v_old_status='available' THEN

            UPDATE `room_instance`

            SET status=CASE WHEN NEW.room_status='checked_in' THEN 'occupied' ELSE 'r
eserved' END,

            updated_at=NOW()

            WHERE id=NEW.room_id;

            INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,sour
ce_id,operator_id,remark)

            VALUES(NEW.room_id,v_old_status,CASE WHEN NEW.room_status='checked_in' THEN
'occupied' ELSE 'reserved' END,'booking_room',NEW.id,NULL,'booking room detail locked ro
om');

        END IF;

    END IF;

END$$

```



```
CREATE TRIGGER `trg_checkin_after_insert_room_occupied`
AFTER INSERT ON `checkin_record`
FOR EACH ROW
BEGIN
    DECLARE v_old_status varchar(30);
    SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_id;
    UPDATE `room_instance` SET status='occupied',updated_at=NOW() WHERE id=NEW.room_id;
    INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_id,operator_id,remark)
VALUES(NEW.room_id,v_old_status,'occupied','checkin_record',NEW.id,NEW.employee_id,'
guest checked in');
    IF NEW.booking_id IS NOT NULL THEN
        UPDATE `booking_room` SET room_status='checked_in' WHERE booking_id=NEW.booking_
id AND room_id=NEW.room_id;
        INSERT INTO `booking_status_log`(booking_id,old_status,new_status,operator_id,remark)
VALUES(NEW.booking_id,NULL,'checked_in',NEW.employee_id,'checkin record inserted
');
    END IF;
    INSERT INTO `operation_log`(operator_type,operator_id,business_type,business_id,action,remark)
VALUES('employee',NEW.employee_id,'checkin_record',NEW.id,'checkin_inserted','room o
ccupied');
END$$

CREATE TRIGGER `trg_checkin_after_update_room_available`
AFTER UPDATE ON `checkin_record`
FOR EACH ROW
BEGIN
    DECLARE v_old_status varchar(30);
    IF OLD.status<>NEW.status AND NEW.status='checked_out' THEN
        SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_id;
        UPDATE `room_instance` SET status='cleaning',updated_at=NOW() WHERE id=NEW.room_
id;
        INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_id,operator_id,remark)
VALUES(NEW.room_id,v_old_status,'cleaning','checkin_record',NEW.id,NEW.employee_
id,'guest checked out and room waits for cleaning');
        IF NEW.booking_id IS NOT NULL THEN
            UPDATE `booking_order` SET booking_status='completed' WHERE id=NEW.booking_i
d;
            UPDATE `booking_room` SET room_status='completed' WHERE booking_id=NEW.booki
ng_id AND room_id=NEW.room_id;
        END IF;
    END IF;
```



```
INSERT INTO `housekeeping_task`(room_id,checkin_id,assigned_employee_id,task_type,task_status,scheduled_time,remark)
VALUES(NEW.room_id,NEW.id,NEW.employee_id,'checkout_cleaning','assigned',NOW(),
created by checkout trigger');
INSERT INTO `operation_log`(operator_type,operator_id,business_type,business_id,action,remark)
VALUES('employee',NEW.employee_id,'checkin_record',NEW.id,'checkout_updated','room cleaning');
END IF;
END$$

CREATE TRIGGER `trg_housekeeping_after_update_release_room`
AFTER UPDATE ON `housekeeping_task`
FOR EACH ROW
BEGIN
DECLARE v_old_status varchar(30);
IF OLD.task_status<>NEW.task_status AND NEW.task_status='completed' THEN
SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_id;
UPDATE `room_instance` SET status='available',updated_at=NOW() WHERE id=NEW.room_id;
INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_id,operator_id,remark)
VALUES(NEW.room_id,v_old_status,'available','housekeeping_task',NEW.id,NEW.assigned_employee_id,'housekeeping completed and room released');
INSERT INTO `operation_log`(operator_type,operator_id,business_type,business_id,action,remark)
VALUES('employee',NEW.assigned_employee_id,'housekeeping_task',NEW.id,'complete_housekeeping','room available');
END IF;
END$$

CREATE TRIGGER `trg_maintenance_after_insert_mark_room`
AFTER INSERT ON `maintenance_record`
FOR EACH ROW
BEGIN
DECLARE v_old_status varchar(30);
IF NEW.maintenance_status IN ('reported','processing') THEN
SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_id;
UPDATE `room_instance` SET status='maintenance',updated_at=NOW() WHERE id=NEW.room_id;
INSERT INTO `room_status_log`(room_id,old_status,new_status,source_type,source_id,operator_id,remark)
VALUES(NEW.room_id,v_old_status,'maintenance','maintenance_record',NEW.id,NEW.reported_by_employee_id,'maintenance reported');
```



```
INSERT INTO `operation_log` (operator_type,operator_id,business_type,business_id,
action,remark)
VALUES('employee',NEW.reported_by_employee_id,'maintenance_record',NEW.id,'maint
enance_reported',NEW.issue_desc);
END IF;
END$$

CREATE TRIGGER `trg_maintenance_after_update_release_room`
AFTER UPDATE ON `maintenance_record`
FOR EACH ROW
BEGIN
DECLARE v_old_status varchar(30);
IF OLD.maintenance_status<>NEW.maintenance_status AND NEW.maintenance_status='comple
ted' THEN
SELECT status INTO v_old_status FROM `room_instance` WHERE id=NEW.room_id;
UPDATE `room_instance` SET status='cleaning',updated_at=NOW() WHERE id=NEW.room_
id;
INSERT INTO `room_status_log` (room_id,old_status,new_status,source_type,source_i
d,operator_id,remark)
VALUES(NEW.room_id,v_old_status,'cleaning','maintenance_record',NEW.id,NEW.handl
ed_by_employee_id,'maintenance completed and room waits for cleaning');
INSERT INTO `housekeeping_task` (room_id,assigned_employee_id,task_type,task_stat
us,scheduled_time,remark)
VALUES(NEW.room_id,NEW.handled_by_employee_id,'maintenance_cleaning','assigned',
NOW(),'created after maintenance completed');
INSERT INTO `operation_log` (operator_type,operator_id,business_type,business_id,
action,remark)
VALUES('employee',NEW.handled_by_employee_id,'maintenance_record',NEW.id,'mainte
nance_completed','room cleaning');
END IF;
END$$

CREATE TRIGGER `trg_payment_after_insert_update_bill`
AFTER INSERT ON `payment_record`
FOR EACH ROW
BEGIN
DECLARE v_payable decimal(10,2);
DECLARE v_paid decimal(10,2);
DECLARE v_new_status varchar(30);
IF NEW.pay_status='success' THEN
SELECT payable_amount,paid_amount+NEW.pay_amount INTO v_payable,v_paid FROM `bil
l` WHERE id=NEW.bill_id;
IF v_paid>=v_payable THEN SET v_new_status='paid';
ELSEIF v_paid>0 THEN SET v_new_status='partial';
```




```
ELSE SET v_new_status='unpaid'; END IF;
UPDATE `bill` SET paid_amount=v_paid,status=v_new_status,updated_at=NOW() WHERE
id=NEW.bill_id;
INSERT INTO `operation_log` (operator_type,operator_id,business_type,business_id,
action,remark)
VALUES('system',NULL,'payment_record',NEW.id,'payment_inserted',NEW.payment_no);
END IF;
END$$

CREATE TRIGGER `trg_refund_after_insert_update_payment`
AFTER INSERT ON `refund_record`
FOR EACH ROW
BEGIN
    DECLARE v_bill_id bigint;
    DECLARE v_total_refund decimal(10,2);
    DECLARE v_paid_amount decimal(10,2);
    DECLARE v_new_status varchar(30);
    IF NEW.refund_status='success' THEN
        SELECT bill_id INTO v_bill_id FROM `payment_record` WHERE id=NEW.payment_id;
        SELECT COALESCE(SUM(rr.refund_amount),0) INTO v_total_refund
        FROM `refund_record` rr JOIN `payment_record` pr ON pr.id=rr.payment_id
        WHERE pr.bill_id=v_bill_id AND rr.refund_status='success';
        SELECT paid_amount INTO v_paid_amount FROM `bill` WHERE id=v_bill_id;
        IF v_total_refund>=v_paid_amount THEN SET v_new_status='refunded'; ELSE SET v_ne
w_status='partial'; END IF;
        UPDATE `bill` SET refund_amount=v_total_refund,status=v_new_status,updated_at=NO
W() WHERE id=v_bill_id;
        INSERT INTO `operation_log` (operator_type,operator_id,business_type,business_id,
action,remark)
        VALUES('system',NULL,'refund_record',NEW.id,'refund_inserted',NEW.refund_no);
    END IF;
END$$

CREATE TRIGGER `trg_room_review_after_insert_update_score`
AFTER INSERT ON `room_review`
FOR EACH ROW
BEGIN
    DECLARE v_review_count int DEFAULT 0;
    DECLARE v_avg_score decimal(4,2) DEFAULT 0.00;
    SELECT COUNT(*),ROUND(AVG(score),2) INTO v_review_count,v_avg_score FROM `room_revie
w` WHERE refid=NEW.refid;
    UPDATE `room_catalog` SET review_count=v_review_count,avg_score=COALESCE(v_avg_score,
0.00) WHERE id=NEW.refid;
```



```

INSERT INTO `operation_log` (operator_type,operator_id,business_type,business_id,action,remark)
VALUES('customer',NEW.userid,'room_review',NEW.id,'room_review_inserted',CONCAT('score=',NEW.score));
END$$

CREATE TRIGGER `trg_hotel_review_after_insert_update_score`
AFTER INSERT ON `hotel_review`
FOR EACH ROW
BEGIN
    DECLARE v_review_count int DEFAULT 0;
    DECLARE v_avg_score decimal(4,2) DEFAULT 0.00;
    SELECT COUNT(*),ROUND(AVG(score),2) INTO v_review_count,v_avg_score FROM `hotel_review` WHERE refid=NEW.refid;
    UPDATE `hotel_profile` SET review_count=v_review_count,avg_score=COALESCE(v_avg_score,0.00) WHERE id=NEW.refid;
    INSERT INTO `operation_log` (operator_type,operator_id,business_type,business_id,action,remark)
    VALUES('customer',NEW.userid,'hotel_review',NEW.id,'hotel_review_inserted',CONCAT('score=',NEW.score));
END$$

DELIMITER ;

```

6.3 子模式（视图）的设计

视图名	作用	主要来源表
v_room_inventory_overview	按客房目录汇总总房间数、空闲数、预订数、在住数、清扫数、维修数	room_catalog、room_instance
v_room_daily_status	展示每间实体客房当前状态、关联订单、入住、清扫、维修信息	room_instance、booking_room、booking_order、checkin_record、housekeeping_task、maintenance_record
v_booking_detail	展示预订订单与顾客、房型、房间明细的综合结果	booking_order、customer、room_catalog、booking_room、room_instance
v_checkin_guest_detail	展示入住记录、入住人信息、办理员工和房间信息	checkin_record、checkin_guest、employee、room_instance、room_catalog
v_bill_payment_summary	展示账单、支付记录、应收	bill、payment_record、

视图名	作用	主要来源表
ary	实收和支付状态	checkin_record、booking_order
v_monthly_revenue_report	按月份统计订单收入、账单收入和实收金额	booking_order、bill、payment_record
v_employee_operation_summary	汇总员工在入住、清扫、维修、账务等流程中的业务量	employee、checkin_record、housekeeping_task、maintenance_record、bill、payment_record、operation_log
v_customer_value_profile	统计顾客订单数、入住次数、消费金额与客户活跃情况	customer、booking_order、checkin_record、bill、payment_record
v_housekeeping_task_detail	展示清扫任务状态、房号、房型和执行员工信息	housekeeping_task、room_instance、room_catalog、employee
v_employee_performance_detail	展示员工考核周期、各项业务指标、总分与等级	employee_performance、employee

表 6 视图汇总一览

以下对于每个视图的实现代码段以及最终呈现进行展示：

1. 客房库存总览视图 (v_room_inventory_overview)

```
CREATE VIEW `v_room_inventory_overview` AS
SELECT rc.id AS room_catalog_id, rc.jiudianmingcheng AS hotel_name, rc.kefangmingcheng AS room_name,
       rc.room_category, COUNT(ri.id) AS total_rooms,
       SUM(CASE WHEN ri.status='available' THEN 1 ELSE 0 END) AS available_rooms,
       SUM(CASE WHEN ri.status='reserved' THEN 1 ELSE 0 END) AS reserved_rooms,
       SUM(CASE WHEN ri.status='occupied' THEN 1 ELSE 0 END) AS occupied_rooms,
       SUM(CASE WHEN ri.status='cleaning' THEN 1 ELSE 0 END) AS cleaning_rooms,
       SUM(CASE WHEN ri.status='maintenance' THEN 1 ELSE 0 END) AS maintenance_rooms,
       ROUND(rc.kefangjiage,2) AS base_price, rc.review_count, rc.avg_score
FROM `room_catalog` rc LEFT JOIN `room_instance` ri ON ri.room_catalog_id=rc.id
GROUP BY rc.id, rc.jiudianmingcheng, rc.kefangmingcheng, rc.room_category, rc.kefangjiage, rc.review_count, rc.avg_score;
```

客房库存总览视图

数据库: hotel_management 视图: v_room_inventory_overview 记录数: 7

room_catalog_id	hotel_name	room_name	room_category	total_rooms	available_rooms	reserved_rooms	occupied_rooms	cleaning_rooms	maintenance_rooms	base_price	review_count	avg_score
3001	ShangriLa Hotel	RoomName1	RoomType1	3	3	0	0	0	0	122	0	0.00
3002	ShangriLa Hotel	RoomName2	RoomType2	2	0	2	0	0	0	2	0	0.00
3003	ShangriLa Hotel	RoomName3	RoomType3	3	2	0	1	0	0	3	0	0.00
3004	ShangriLa Hotel	RoomName4	RoomType4	4	1	3	0	0	0	4	0	0.00
3005	ShangriLa Hotel	RoomName5	RoomType5	5	4	0	0	1	0	5	0	0.00
3006	ShangriLa Hotel	RoomName6	RoomType6	6	4	1	0	1	0	6	0	0.00
3007	ShangriLa Hotel	river view room	DeluxeSuite	10	7	2	0	0	1	666	0	0.00

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 7 客房库存视图

2. 客房日常状态视图 (v_room_daily_status)

```
CREATE VIEW `v_room_daily_status` AS
SELECT ri.id AS room_id, ri.room_no, ri.floor_no, ri.status AS current_status,
```

```
rc.kefangmingcheng AS room_name, rc.room_category, br.booking_id AS
active_booking_id,
bo.yudingbianhao AS booking_no, bo.customerming AS customer_name,
cr.id AS active_checkin_id, cr.checkin_time, cr.checkout_time,
ht.id AS active_housekeeping_task_id, mr.id AS active_maintenance_id,
CASE WHEN cr.id IS NOT NULL AND cr.status='checked_in' THEN 'occupied_by_checkin'
      WHEN mr.id IS NOT NULL AND mr.maintenance_status IN ('reported','processing') THEN
'maintenance_processing'
      WHEN ht.id IS NOT NULL AND ht.task_status IN ('pending','assigned','in_progress')
THEN 'cleaning_processing'
      WHEN br.id IS NOT NULL AND br.room_status IN ('reserved','checked_in') THEN
'reserved_by_booking'
      ELSE ri.status END AS business_status
FROM `room_instance` ri JOIN `room_catalog` rc ON rc.id=ri.room_catalog_id
LEFT JOIN `booking_room` br ON br.room_id=ri.id AND br.room_status IN
('reserved','checked_in')
LEFT JOIN `booking_order` bo ON bo.id=br.booking_id AND bo.booking_status IN
('created','reserved','checked_in')
LEFT JOIN `checkin_record` cr ON cr.room_id=ri.id AND cr.status='checked_in'
LEFT JOIN `housekeeping_task` ht ON ht.room_id=ri.id AND ht.task_status IN
('pending','assigned','in_progress')
LEFT JOIN `maintenance_record` mr ON mr.room_id=ri.id AND mr.maintenance_status IN
('reported','processing');
```

客房日常状态视图

数据库: hotel_management 视图: v_room_daily_status 记录数: 33

room_id	room_no	floor_no	current_status	room_name	room_category	active_booking_id	booking_no	customer_name	active_checkin_id	checkin_time	checkout_time	active_housekeeping...	active_maintenance...	business_status
8007	R0101	1	available	RoomName1	RoomType1	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8014	R0102	1	available	RoomName1	RoomType1	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8020	R0103	1	available	RoomName1	RoomType1	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8006	R0201	1	reserved	RoomName2	RoomType2	6003	BK202604290003	user3	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8013	R0202	1	reserved	RoomName2	RoomType2	6005	BK202604290005	user5	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8005	R0301	1	available	RoomName3	RoomType3	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8012	R0302	1	available	RoomName3	RoomType3	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8019	R0303	1	occupied	RoomName3	RoomType3	6009	BK202605080009	user2	9302	2026-05-08 10:00:00	NULL	NULL	NULL	occupied_by_checkin
8004	R0401	1	reserved	RoomName4	RoomType4	6002	BK202604290002	user2	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8011	R0402	1	available	RoomName4	RoomType4	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8018	R0403	1	reserved	RoomName4	RoomType4	6006	BK202604290006	user6	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8024	R0404	1	reserved	RoomName4	RoomType4	6007	BK202604290007	user7	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8003	R0501	1	cleaning	RoomName5	RoomType5	NULL	NULL	NULL	NULL	NULL	NULL	9701	NULL	cleaning_processing
8010	R0502	1	available	RoomName5	RoomType5	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8017	R0503	1	available	RoomName5	RoomType5	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8023	R0504	1	available	RoomName5	RoomType5	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8027	R0505	1	available	RoomName5	RoomType5	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8002	R0601	1	cleaning	RoomName6	RoomType6	NULL	NULL	NULL	NULL	NULL	NULL	9702	NULL	cleaning_processing
8009	R0602	1	reserved	RoomName6	RoomType6	6004	BK202604290004	user4	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8016	R0603	1	available	RoomName6	RoomType6	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8022	R0604	1	available	RoomName6	RoomType6	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8026	R0605	1	available	RoomName6	RoomType6	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8029	R0606	1	available	RoomName6	RoomType6	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8001	R0701	1	reserved	river view room	DeluxeSuite	6001	BK202604290001	user1	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8008	R0702	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8015	R0703	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8021	R0704	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8025	R0705	1	reserved	river view room	DeluxeSuite	6007	BK202604290007	user7	NULL	NULL	NULL	NULL	NULL	reserved_by_booking
8028	R0706	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8030	R0707	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8031	R0708	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8032	R0709	1	available	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	available
8033	R0710	1	maintenance	river view room	DeluxeSuite	NULL	NULL	NULL	NULL	NULL	NULL	NULL	9801	maintenance_processing

当前视图由本地 hotel_management 数据库中的演示数据自动生成。

图 8 客房日常状态视图

3. 预定明细视图 (v_booking_detail)

```
CREATE VIEW `v_booking_detail` AS
```

```

SELECT bo.id AS booking_id, bo.yudingbianhao AS booking_no, bo.addtime AS booking_time,
       bo.customerming AS customer_login_name, COALESCE(bo.customer_id,c.id) AS customer_id,
       c.xingming AS customer_real_name,
       c.shoujihao AS customer_phone, bo.kefangmingcheng AS room_name, bo.room_category,
       GROUP_CONCAT(DISTINCT ri.room_no ORDER BY ri.room_no SEPARATOR ',') AS room_numbers,
       bo.expected_checkin_date, bo.expected_checkout_date,
       bo.kefangjiage AS unit_price, bo.shuliang AS quantity, bo.zongjine AS total_amount,
       bo.ispay AS old_payment_status, bo.booking_status, COUNT(DISTINCT bs1.id) AS
status_change_count,
       MAX(bs1.created_at) AS last_status_time
FROM `booking_order` bo
LEFT JOIN `customer` c ON c.id=bo.customer_id OR (bo.customer_id IS NULL AND
c.customerming=bo.customerming)
LEFT JOIN `booking_room` br ON br.booking_id=bo.id
LEFT JOIN `room_instance` ri ON ri.id=br.room_id
LEFT JOIN `booking_status_log` bs1 ON bs1.booking_id=bo.id
GROUP BY bo.id, bo.yudingbianhao, bo.addtime, bo.customerming, bo.customer_id, c.id,
c.xingming, c.shoujihao,
       bo.kefangmingcheng, bo.room_category, bo.expected_checkin_date,
bo.expected_checkout_date,
       bo.kefangjiage, bo.shuliang, bo.zongjine, bo.ispay, bo.booking_status;
  
```

预订明细视图

数据库: hotel_management 视图: v_booking_detail 记录数: 9

booking_id	booking_no	booking_time	customer_login_n	customer_id	customer_real_n	customer_phc	room_name	room_cate	room_numb	expected_checkin	expected_checkout	unit_price	quantity	total_amount	old_payment_status	booking_status	change_cc	last_status
6001	BK20260429000206-04-29 18:00:01	2026-04-29 18:00:02	4001	Customer1	13823888881	RoomName1	RoomType1	R0701	2026-04-29	2026-04-30	122	1	122	unpaid	created	0	NULL	
6002	BK20260429000206-04-29 18:00:02		4002	Customer2	13823888882	RoomName2	RoomType2	R0401	2026-04-29	2026-04-30	2	1	2	unpaid	created	0	NULL	
6003	BK20260429000206-04-29 18:00:03		4003	Customer3	13823888883	RoomName3	RoomType3	R0201	2026-04-29	2026-04-30	3	1	3	unpaid	created	0	NULL	
6004	BK20260429000206-04-29 18:00:04		4004	Customer4	13823888884	RoomName4	RoomType4	R0602	2026-04-29	2026-04-30	4	1	4	unpaid	created	0	NULL	
6005	BK20260429000206-04-29 18:20:05	2026-04-29 18:25:06	4005	Customer5	13823888885	RoomName5	RoomType5	R0202	2026-04-29	2026-04-30	5	1	5	unpaid	created	0	NULL	
6006	BK20260429000206-04-29 18:25:06		4006	Customer6	13823888886	RoomName6	RoomType6	R0403	2026-04-29	2026-04-30	6	1	6	unpaid	created	0	NULL	
6007	BK20260429000206-04-29 18:30:07		4007	HuYue	13612512514	river view roomDeluxeSuite	R0404,R0705	2026-04-29	2026-04-30	666	2	1332	paid	created	0	NULL		
6008	BK20260508000206-05-08 18:00:01	2026-05-08 18:00:02	4001	Customer1	13823888881	RoomName1	RoomType1	R0601	2026-05-08	2026-05-09	122	1	122	unpaid	completed	5	2026-05-08	
6009	BK20260508000206-05-08 18:00:02		4002	Customer2	13823888882	RoomName6	RoomType6	R0303	2026-05-08	2026-05-09	6	1	6	unpaid	checked_in	4	2026-05-08	

当前视图由本地 hotel_management 数据库的演示数据自动生成。

图 9 预定明细视图

4. 入住客人明细视图 (v_checkin_guest_detail)

```

CREATE VIEW `v_checkin_guest_detail` AS
SELECT cr.id AS checkin_id, cr.booking_id, cr.customer_id, c.xingming AS customer_name,
       ri.room_no, rc.kefangmingcheng AS room_name, rc.room_category, cr.checkin_time,
       cr.checkout_time, cr.status AS checkin_status,
       MAX(CASE WHEN cg.is_primary=1 THEN cg.guest_name END) AS primary_guest_name,
       COUNT(cg.id) AS guest_count, e.employeexingming AS employee_name
FROM `checkin_record` cr JOIN `room_instance` ri ON ri.id=cr.room_id JOIN `room_catalog` rc
ON rc.id=ri.room_catalog_id
LEFT JOIN `customer` c ON c.id=cr.customer_id LEFT JOIN `employee` e ON e.id=cr.employee_id
LEFT JOIN `checkin_guest` cg ON cg.checkin_id=cr.id
GROUP BY cr.id, cr.booking_id, cr.customer_id, c.xingming, ri.room_no, rc.kefangmingcheng,
rc.room_category,
       cr.checkin_time, cr.checkout_time, cr.status, e.employeexingming;
  
```

入住客人明细视图

数据库: hotel_management 视图: v_checkin_guest_detail 记录数: 2

checkin_id	booking_id	customer_id	customer_name	room_no	room_name	room_category	checkin_time	checkout_time	checkin_status	primary_guest_name	guest_count	employee_name
9301	6008	4001	Customer1	R0601	RoomName6	RoomType6	2026-05-08 10:00:00	2026-05-08 10:00:00	checked_out	Customer1	1	Employee1
9302	6009	4002	Customer2	R0303	RoomName3	RoomType3	2026-05-08 10:00:00	NULL	checked_in	Customer2	1	Employee2

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 10 入住客人明细视图

5. 账单支付汇总视图 (v_bill_payment_summary)

```

CREATE VIEW `v_bill_payment_summary` AS
SELECT b.id AS bill_id, b.booking_id, b.checkin_id, b.customer_id, c.xingming AS
customer_name,
      b.room_amount, b.service_amount, b.discount_amount, b.payable_amount,
      COALESCE(SUM(CASE WHEN pr.pay_status='success' THEN pr.pay_amount ELSE 0 END),0) AS
actual_paid_amount,
      COALESCE(SUM(CASE WHEN rr.refund_status='success' THEN rr.refund_amount ELSE 0 END),0)
AS actual_refund_amount,
      b.status AS bill_status, MAX(pr.paid_at) AS last_paid_at, b.created_at
FROM `bill` b LEFT JOIN `customer` c ON c.id=b.customer_id
LEFT JOIN `payment_record` pr ON pr.bill_id=b.id LEFT JOIN `refund_record` rr ON
rr.payment_id=pr.id
GROUP BY b.id, b.booking_id, b.checkin_id, b.customer_id, c.xingming, b.room_amount,
b.service_amount,
      b.discount_amount, b.payable_amount, b.status, b.created_at;
  
```

账单支付汇总视图

数据库: hotel_management 视图: v_bill_payment_summary 记录数: 1

bill_id	booking_id	checkin_id	customer_id	customer_name	room_amount	service_amount	discount_amount	payable_amount	actual_paid_amount	actual_refund_amount	bill_status	last_paid_at	created_at
9501	6008	9301	4001	Customer1	122.00	20.00	5.00	137.00	137.00	0.00	paid	2026-05-08 10:00:00	2026-05-08 10:00:00

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 11 账单支付汇总视图

6. 月度营收统计视图 (v_monthly_revenue_report)

```

CREATE VIEW `v_monthly_revenue_report` AS
SELECT DATE_FORMAT(pr.paid_at,'%Y-%m') AS revenue_month, rc.room_category, pr.pay_method,
      COUNT(DISTINCT b.id) AS bill_count, COUNT(pr.id) AS payment_count,
      SUM(CASE WHEN pr.pay_status='success' THEN pr.pay_amount ELSE 0 END) AS gross_revenue,
      COALESCE(SUM(CASE WHEN rr.refund_status='success' THEN rr.refund_amount ELSE 0 END),0)
AS refund_amount,
      SUM(CASE WHEN pr.pay_status='success' THEN pr.pay_amount ELSE 0
END)-COALESCE(SUM(CASE WHEN rr.refund_status='success' THEN rr.refund_amount ELSE 0 END),0)
AS net_revenue
FROM `payment_record` pr JOIN `bill` b ON b.id=pr.bill_id
LEFT JOIN `checkin_record` cr ON cr.id=b.checkin_id LEFT JOIN `room_instance` ri ON
ri.id=cr.room_id
LEFT JOIN `room_catalog` rc ON rc.id=ri.room_catalog_id LEFT JOIN `refund_record` rr ON
rr.payment_id=pr.id
  
```

GROUP BY DATE_FORMAT(pr.paid_at,'%Y-%m'), rc.room_category, pr.pay_method;

月度营收统计视图

数据库: hotel_management 视图: v_monthly_revenue_report 记录数: 1

revenue_month	room_category	pay_method	bill_count	payment_count	gross_revenue	refund_amount	net_revenue
2026-05	RoomType6	wechat	1	1	137.00	0.00	137.00

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 12 月度营收统计视图

7. 员工业务汇总视图 (v_employee_operation_summary)

```
CREATE VIEW `v_employee_operation_summary` AS
SELECT e.id AS employee_id, e.employeegonghao AS employee_no, e.employeexingming AS
employee_name,
      COUNT(DISTINCT ol.id) AS total_operations,
      COUNT(DISTINCT CASE WHEN ol.business_type='booking_order' THEN ol.id END) AS
booking_operations,
      COUNT(DISTINCT CASE WHEN ol.business_type='checkin_record' THEN ol.id END) AS
checkin_operations,
      COUNT(DISTINCT CASE WHEN ol.business_type='bill' THEN ol.id END) AS bill_operations,
      COUNT(DISTINCT CASE WHEN ol.business_type='refund_record' THEN ol.id END) AS
refund_operations,
      COUNT(DISTINCT ht.id) AS cleaning_task_count,
      COUNT(DISTINCT mr.id) AS maintenance_task_count,
      MAX(ep.total_score) AS latest_assessment_score,
      MAX(ep.grade) AS latest_assessment_grade,
      MIN(ol.created_at) AS first_operation_time, MAX(ol.created_at) AS
last_operation_time
FROM `employee` e LEFT JOIN `operation_log` ol ON ol.operator_type='employee' AND
ol.operator_id=e.id
LEFT JOIN `housekeeping_task` ht ON ht.assigned_employee_id=e.id AND
ht.task_status='completed'
LEFT JOIN `maintenance_record` mr ON mr.handled_by_employee_id=e.id AND
mr.maintenance_status='completed'
LEFT JOIN `employee_performance` ep ON ep.employee_id=e.id
GROUP BY e.id, e.employeegonghao, e.employeexingming;
```

员工业务汇总视图

数据库: hotel_management 视图: v_employee_operation_summary 记录数: 7

employee_id	employee_no	employee_name	total_operations	booking_operations	checkin_operations	bill_operations	refund_operations	cleaning_task_count	maintenance_task_co...	latest_assessment_s...	latest_assessment_g...	first_operation_time	last_operation_time
S001	EMP001	Employee1	5	0	3	1	0	0	0	32.00	D	2026-05-08 10:00:00	2026-05-08 10:00:00
S002	EMP002	Employee2	2	0	2	0	0	0	0	19.00	D	2026-05-08 10:00:00	2026-05-08 10:00:00
S003	EMP003	Employee3	0	0	0	0	0	0	0	NULL	NULL	NULL	NULL
S004	EMP004	Employee4	0	0	0	0	0	0	0	NULL	NULL	NULL	NULL
S005	EMP005	Employee5	0	0	0	0	0	0	0	NULL	NULL	NULL	NULL
S006	EMP006	Employee6	0	0	0	0	0	0	0	NULL	NULL	NULL	NULL
S007	EMP007	HuYue	0	0	0	0	0	0	0	NULL	NULL	NULL	NULL

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 13 员工业务汇总视图

8. 客户价值画像视图 (v_customer_value_profile)


```

CREATE VIEW `v_customer_value_profile` AS
SELECT c.id AS customer_id, c.customerming AS customer_login_name, c.xingming AS
customer_name, c.shoujihao AS customer_phone,
      COUNT(DISTINCT bo.id) AS booking_count, COUNT(DISTINCT cr.id) AS checkin_count,
      COALESCE(SUM(DISTINCT b.payable_amount),0) AS total_payable_amount,
      COALESCE(SUM(DISTINCT b.paid_amount),0) AS total_paid_amount,
      COUNT(DISTINCT f.id) AS favorite_count, COUNT(DISTINCT rr.id) AS room_review_count,
      COUNT(DISTINCT hr.id) AS hotel_review_count, MAX(bo.addtime) AS last_booking_time
FROM `customer` c LEFT JOIN `booking_order` bo ON bo.customerming=c.customerming
LEFT JOIN `checkin_record` cr ON cr.customer_id=c.id LEFT JOIN `bill` b ON b.customer_id=c.id
LEFT JOIN `favorite` f ON f.userid=c.id LEFT JOIN `room_review` rr ON rr.userid=c.id LEFT
JOIN `hotel_review` hr ON hr.userid=c.id
GROUP BY c.id, c.customerming, c.xingming, c.shoujihao;
  
```

客户价值画像视图

数据库: hotel_management 视图: v_customer_value_profile 记录数: 7

customer_id	customer_login_name	customer_name	customer_phone	booking_count	checkin_count	total_payable_amount	total_paid_amount	favorite_count	room_review_count	hotel_review_count	last_booking_time
4001	user1	Customer1	13823888881	2	1	137.00	137.00	2	1	1	2026-05-08 18:00:00
4002	user2	Customer2	13823888882	2	1	0.00	0.00	0	1	1	2026-05-08 18:00:00
4003	user3	Customer3	13823888883	1	0	0.00	0.00	0	1	1	2026-04-29 18:10:00
4004	user4	Customer4	13823888884	1	0	0.00	0.00	0	1	1	2026-04-29 18:15:00
4005	user5	Customer5	13823888885	1	0	0.00	0.00	0	1	1	2026-04-29 18:20:00
4006	user6	Customer6	13823888886	1	0	0.00	0.00	0	1	1	2026-04-29 18:25:00
4007	user7	HuYue	13612512514	1	0	0.00	0.00	2	1	1	2026-04-29 18:30:00

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 14 客户价值画像视图

9. 客房清扫任务视图 (v_housekeeping_task_detail)

```

CREATE VIEW `v_housekeeping_task_detail` AS
SELECT ht.id AS task_id, ht.task_type, ht.task_status, ht.scheduled_time, ht.completed_time,
      ri.room_no, ri.status AS room_status, rc.kefangmingcheng AS room_name,
      rc.room_category,
      cr.booking_id, e.employeexingming AS assigned_employee_name, ht.remark,
      ht.created_at
FROM `housekeeping_task` ht
JOIN `room_instance` ri ON ri.id=ht.room_id
JOIN `room_catalog` rc ON rc.id=ri.room_catalog_id
LEFT JOIN `checkin_record` cr ON cr.id=ht.checkin_id
LEFT JOIN `employee` e ON e.id=ht.assigned_employee_id;
  
```

客房清扫任务视图

数据库: hotel_management 视图: v_housekeeping_task_detail 记录数: 2

task_id	task_type	task_status	scheduled_time	completed_time	room_no	room_status	room_name	room_category	booking_id	assigned_employee_n...	remark	created_at
9701	daily_cleaning	assigned	2026-05-07 09:20:00	NULL	R0501	cleaning	RoomName5	RoomType5	NULL	Employee1	sample housekeeping task	2026-05-07 09:20:00
9702	checkout_cleaning	assigned	2026-05-08 10:00:00	NULL	R0601	cleaning	RoomName6	RoomType6	6008	Employee1	created by checkout trigger	2026-05-08 10:00:00

当前截图由本地 hotel_management 数据库的演示数据自动生成。

图 15 客房清扫任务视图

10. 员工考核明细视图 (v_employee_performance_detail)

```

CREATE VIEW `v_employee_performance_detail` AS
SELECT ep.id AS performance_id, ep.employee_id, e.employeegonghao AS employee_no,
      e.employeexingming AS employee_name, ep.period_start, ep.period_end,
  
```



```

ep.checkin_count, ep.bill_count, ep.payment_count, ep.cleaning_count,
ep.maintenance_count, ep.operation_count, ep.manager_score, ep.total_score,
ep.grade, assessor.employeeexingming AS assessor_name, ep.created_at
FROM `employee_performance` ep
JOIN `employee` e ON e.id=ep.employee_id
LEFT JOIN `employee` assessor ON assessor.id=ep.assessor_id;

```

员工考核明细视图

数据库: hotel_management 视图: v_employee_performance_detail 记录数: 2

performance_id	employee_id	employee_no	employee_name	period_start	period_end	checkin_count	bill_count	payment_count	cleaning_count	maintenance_count	operation_count	manager_score	total_score	grade	assessor_name	created_at
9901	5001	EMP001	Employee1	2026-05-01	2026-05-08	1	1	1	0	0	5	15.00	32.00	D	Employee2	2026-05-08 10:00:00
9902	5002	EMP002	Employee2	2026-05-01	2026-05-08	1	0	0	0	0	2	12.00	19.00	D	Employee1	2026-05-08 10:00:00

当前视图由本地 hotel_management 数据库的演示数据自动生成。

图 16 员工考核明细视图